

Lexical Analysis

Uday Khedker

(www.cse.iitb.ac.in/~uday)

Department of Computer Science and Engineering,
Indian Institute of Technology, Bombay



January 2025



Outline

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

- Introduction
- Specifying scanners
- Tokenizing input using DFAs
- Constructing DFAs
- Representing DFAs using four-arrays
- Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Introduction



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Introduction

Credits:

This part has been adapted from the slides of Prof. Amitabha Sanyal, IIT Bombay



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Introduction

An input program as you see it.

```
main ()
{
    int i,sum;
    sum = 0;
    for (i=1; i<=10; i++)
        sum = sum + i;
    printf("%d\n",sum);
}
```



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:
Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Introduction

The same program as the compiler initially sees it. A continuous sequence of characters without any structure

```
main() {  
    int i, sum; sum = 0;  
    for(i=1; i<=10; i++); sum = sum + i;  
    printf("%d\n", sum);  
}
```

- ┐ The blank space character
- ↵ The return character



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction

Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Introduction

The same program as the compiler initially sees it. A continuous sequence of characters without any structure

```
main()↵{↵int i,sum;↵sum=0;↵  
for(i=1;i<=10;i++);sum=sum+i;↵  
printf("%d\n",sum);↵}
```

How do you make the compiler see what you see?



Discovering the Structure of the Program

Step 1:

- a. Break up this string into the smallest meaningful units.

```
main ( ) {  
    int i , sum  
;  
    sum = 0 ;  
    for ( i = 1 ; i <= 10 ; i ++ )  
    sum = sum + i ;  
    printf ( "%d\n" , sum ) ;  
}
```

We get a sequence of *lexemes* or *tokens*.



Discovering the Structure of the Program

Step 1:

b. During this process, remove the `␣` and the `↵` characters.

```
main ( ) { int i , sum ; sum = 0 ; for (
i = 1 ; i <= 10 ; i ++ ) ; sum = sum + i
; printf ( "%d\n" , sum ) ; }
```

Steps 1a. and 1b. are interleaved.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Discovering the Structure of the Program

Step 1:

b. During this process, remove the `␣` and the `↵` characters.

```
main ( ) { int i , sum ; sum = 0 ; for (
i = 1 ; i <= 10 ; i ++ ) ; sum = sum + i
; printf ( "%d\n" , sum ) ; }
```

This is *lexical analysis* or *scanning*.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Discovering the Structure of the Program

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction

Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Step 2:

Now group the lexemes to form larger structures.

```
main ( ) { int i , sum ; sum = 0 ; for (
i = 1 ; i <= 10 ; i ++ ) ; sum = sum + i
; printf ( "%d\n" , sum ) ; }
```



Discovering the Structure of the Program

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

This is *syntax analysis* or *parsing*.



Discovering the Structure of the Program

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Why is structure finding done in two steps?

- The process of breaking a program into lexemes (scanning) is easier. Use a separate technique to do this.
- Reduces the work to be done by the parser.

However, there are tools (Antlr for example) that indeed combine scanning with parsing.



Lexemes, Tokens and Patterns

Definition: *Lexical analysis* is the operation of dividing the input program into a sequence of *lexemes* (*tokens*).

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

Definition: *Lexical analysis* is the operation of dividing the input program into a sequence of *lexemes* (*tokens*).

Distinguish between

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

Definition: *Lexical analysis* is the operation of dividing the input program into a sequence of *lexemes* (*tokens*).

Distinguish between

- *lexemes* – smallest logical units (words) of a program.

Examples – *i*, *sum*, *for*, *10*, *++*, *"%d\n"*, *<=*.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

Definition: *Lexical analysis* is the operation of dividing the input program into a sequence of *lexemes* (*tokens*).

Distinguish between

- *lexemes* – smallest logical units (words) of a program.
Examples – *i*, *sum*, *for*, *10*, *++*, *"%d\n"*, *<=*.
- *tokens* – sets of similar lexemes, i.e. lexemes which have a common syntactic description.

Examples –

identifier = {*i*, *sum*, *buffer*, ...}

int_constant = {*1*, *10*, ...}

addop = {*+*, *-*}

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

Lexemes which play similar roles during syntax analysis are grouped into a common token.



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

Lexemes which play similar roles during syntax analysis are grouped into a common token.

- Operators in `addop` and `mulop` have different roles – `mulop` has an higher precedence than `addop`.



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

Lexemes which play similar roles during syntax analysis are grouped into a common token.

- Operators in `addop` and `mulop` have different roles – `mulop` has an higher precedence than `addop`.
- Each keyword plays a different role – is therefore a token by itself.



Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

Lexemes which play similar roles during syntax analysis are grouped into a common token.

- Operators in `addop` and `mulop` have different roles – `mulop` has an higher precedence than `addop`.
- Each keyword plays a different role – is therefore a token by itself.
- Each punctuation symbol and each delimiter is a token by itself.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

Lexemes which play similar roles during syntax analysis are grouped into a common token.

- Operators in `addop` and `mulop` have different roles – `mulop` has an higher precedence than `addop`.
- Each keyword plays a different role – is therefore a token by itself.
- Each punctuation symbol and each delimiter is a token by itself.
- All comments are uniformly ignored. They are all grouped under the same token.



Lexemes, Tokens and Patterns

What is the basis for grouping lexemes into tokens?

- Why can't `addop` and `mulop` be combined? Why can't `+` be a token by itself?

Lexemes which play similar roles during syntax analysis are grouped into a common token.

- Operators in `addop` and `mulop` have different roles – `mulop` has an higher precedence than `addop`.
- Each keyword plays a different role – is therefore a token by itself.
- Each punctuation symbol and each delimiter is a token by itself.
- All comments are uniformly ignored. They are all grouped under the same token.
- All identifiers are grouped in a common token.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Lexemes that are not passed to the later stages of a compiler:

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes that are not passed to the later stages of a compiler:

- comments



Lexemes, Tokens and Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes that are not passed to the later stages of a compiler:

- comments
- white spaces – tab, blanks and newlines
 - White spaces are more like separators between lexemes.



Lexemes, Tokens and Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes that are not passed to the later stages of a compiler:

- comments
- white spaces – tab, blanks and newlines
 - White spaces are more like separators between lexemes.

These too have to be detected and then ignored.



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes, Tokens and Patterns

Apart from the token itself, the lexical analyser also passes other information regarding the token. These items of information are called *token attributes*

EXAMPLE

lexeme	<token, token value>
3	< const, 3>
A	<identifier, A>
if	<if, ->
=	<assignop, ->
>	<relop, >>
;	<semicolon, ->



Lexemes, Tokens and Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

The lexical analyser:

- detects the next lexeme
- categorises it into the right token
- passes to the syntax analyser
 - the token name for further syntax analysis
 - the lexeme itself, in some form, for stages beyond syntax analysis



Example – Tokens in Java

1. **Identifier:** A *Javaletter* followed by zero or more *Javaletterordigits*.

A *Javaletter* includes the characters a-z, A-Z, _ and \$.

2. **Constants:**

- 2.1 Integer Constants – 4 byte (usual int) and 8 byte (long int ending with an L) representations.

- Binary – 0b0000011,
- Octal – 027 (Note the leading 0),
- Hex – 0xf28,
- Decimal – 1, -1

- 2.2 Floating point constants

- Float – 1.0345F, 1.04E-12f, .0345f, 1.04e-13f – ends with f or F,
- Double – 5.6E-120D, 123.4d, 0.1 – ends with d or D, or does not end with any of f, F, d, D

- 2.3 Boolean constants – true and false

- 2.4 Character constants – 'a', '\u0034' (Unicode hex), '\t'

- 2.5 String constants – "", "\", "A string".

- 2.6 Null constant – null.



Example – Tokens in Java

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

3. **Delimiters:** (,), {, }, [,] , ;, . and ,
4. **Operators:** =, >, <, ..., >=
5. **Keywords:** abstract, boolean ... volatile, while.



Lexemes, Tokens and Patterns

How does one describe the lexemes that make up the token *identifier*.

Variants in different languages.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lexemes, Tokens and Patterns

How does one describe the lexemes that make up the token *identifier*.

Variants in different languages.

- String of alphanumeric characters. The first character is an alphabet.



Lexemes, Tokens and Patterns

How does one describe the lexemes that make up the token *identifier*.

Variants in different languages.

- String of alphanumeric characters. The first character is an alphabet.
- a string of alphanumeric characters in which the first character is an alphabet. It has a length of at most 31.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

How does one describe the lexemes that make up the token *identifier*.

Variants in different languages.

- String of alphanumeric characters. The first character is an alphabet.
- a string of alphanumeric characters in which the first character is an alphabet. It has a length of at most 31.
- a string of alphabet or numeric or underline characters in which the first character is an alphabet or an underline. It has a length of at most 31. Any character after the 31st are ignored.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

How does one describe the lexemes that make up the token *identifier*.

Variants in different languages.

- String of alphanumeric characters. The first character is an alphabet.
- a string of alphanumeric characters in which the first character is an alphabet. It has a length of at most 31.
- a string of alphabet or numeric or underline characters in which the first character is an alphabet or an underline. It has a length of at most 31. Any character after the 31st are ignored.

Such descriptions are called *patterns*. The description may be informal or formal. *Regular expressions* are the most commonly used formal patterns.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Lexemes, Tokens and Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

A pattern is used to

- *specify tokens* precisely
- *build a recognizer* from such specifications



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

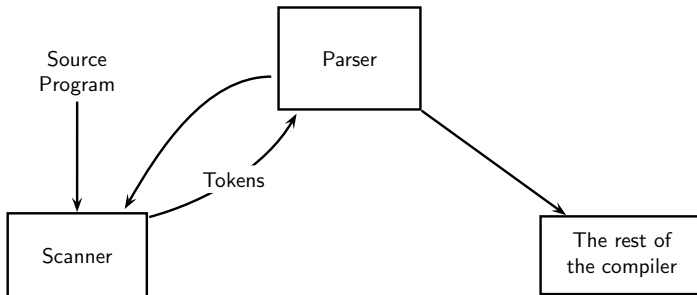
Tokenizing the Input

Representing DFAs

Minimizing DFAs

The Place of a Scanner in a Compiler Organization

- The front end of most compilers is parser driven.
- When the parser needs the next token, it invokes the Lexical Analyser.
- Instead of analysing the entire input string, the lexical analyser sees enough of the input string to return a single token.
- The actions of the lexical analyser and parser are interleaved.





Creating a Lexical Analyzer

IIT Bombay
cs302: Implementation
of Programming
Languages

Two approaches:

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Creating a Lexical Analyzer

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Two approaches:

1. *Hand code* – This is only of historical interest now.
 - Possibly more efficient.



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Creating a Lexical Analyzer

Two approaches:

1. *Hand code* – This is only of historical interest now.
 - Possibly more efficient.
2. *Use a generator* – To generate the lexical analyser from a formal description.
 - The generation process is faster.
 - Less prone to errors.



Automatic Generation of Lexical Analysers

- A **formal description (specification)** of the tokens of the source language, will consist of:
 - a **regular expression** describing each token, and
 - a code fragment called an **action routine** describing the action to be performed, on identifying each token.

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Automatic Generation of Lexical Analysers

- A **formal description (specification)** of the tokens of the source language, will consist of:
 - a **regular expression** describing each token, and
 - a code fragment called an **action routine** describing the action to be performed, on identifying each token.
- Here is a description of **whole numbers** and **identifiers** in form accepted by the lexical analyser generator **Lex**.

action routines

{DIGIT}+

```
{ yylval = atoi(yytext);  
  return NUM;  
}
```

{LETTER}({LETTER}|{DIGIT})*

```
{ yylval = yytext;  
  return IDENTIFIER;  
}
```

(lexeme, token, token value)



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Automatic Generation of Lexical Analysers

- A **formal description (specification)** of the tokens of the source language, will consist of:
 - a **regular expression** describing each token, and
 - a code fragment called an **action routine** describing the action to be performed, on identifying each token.
- Here is a description of **whole numbers** and **identifiers** in form accepted by the lexical analyser generator **Lex**.

{DIGIT}+	<pre>{ yylval = atoi(yytext); return NUM; }</pre>
{LETTER}({LETTER} {DIGIT})*	<pre>{ yylval = yytext; return IDENTIFIER; }</pre>

- The global variable **yylval** holds the token attribute (henceforth to be called token value).



Automatic Generation of Lexical Analysers

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lex can read this description and generate a lexical analyser for whole numbers and identifiers. How?

- The generator puts together:
 - A **deterministic finite automaton (DFA)** constructed from the token specification.
 - A code fragment called a **driver routine** which can traverse **any DFA**.
 - Code for the **action routines**.



Automatic Generation of Lexical Analysers

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Lex can read this description and generate a lexical analyser for whole numbers and identifiers. How?

- The generator puts together:
 - A **deterministic finite automaton (DFA)** constructed from the token specification.
 - A code fragment called a **driver routine** which can traverse **any DFA**.
 - Code for the **action routines**.
- These three things taken together constitutes the **generated lexical analyser**.



Automatic Generation of Lexical Analysers

- How is the lexical analyser generated from the description?

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

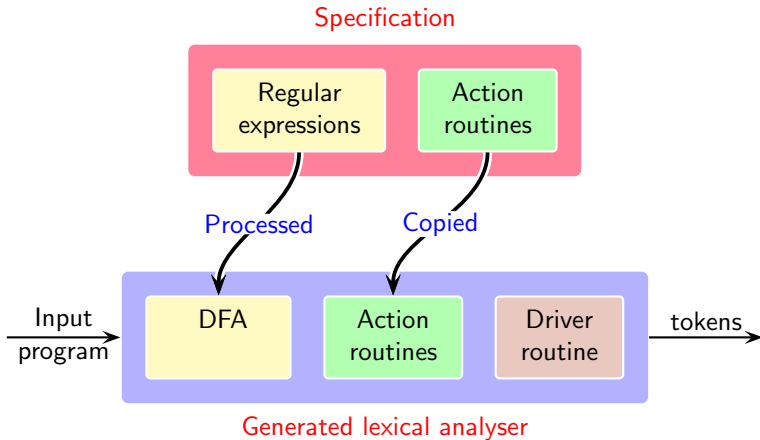
Regular
expressions

Action
routines



Automatic Generation of Lexical Analysers

- How is the lexical analyser generated from the description?



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

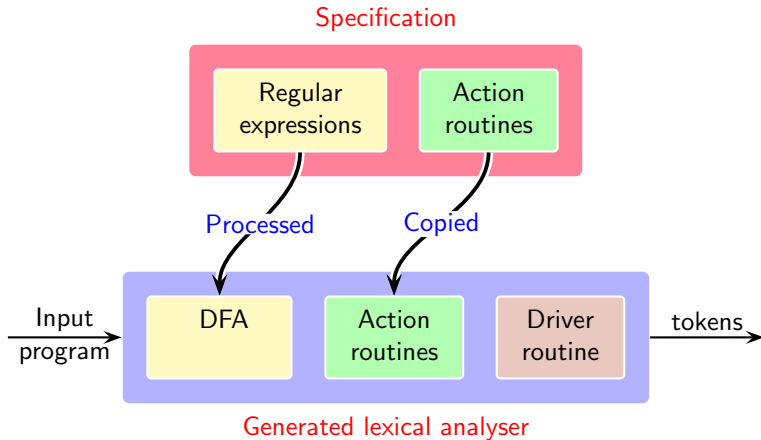


IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Automatic Generation of Lexical Analysers

- How is the lexical analyser generated from the description?



- The driver routine is common for all generated lexical analysers.



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

LEX INPUT FILE FORMAT

Declarations

%%

Patterns
Actions

%%

Auxiliary Code



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Specifying Scanners



Introduction

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Prof. Sanyal's slides (scanning-slides-sanyal-part2.pdf)

Section:
Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Constructing DFAs



Constructing DFA for Multiple Patterns

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

- Join multiple DFAs/NFAs using ϵ transition
Transition without consuming any input symbol
- This creates an NFA (Non-deterministic Finite Automaton)
 - Possible transition without consuming any input symbol
 - Possibly multiple transitions on the same input symbol
- Make the NFA deterministic by subset construction
 - Each state in the resulting DFA is a set of “similar” states of the NFA
 - The start state of the DFA is a union of all original start states (of multiple patterns)
 - Subsequent states are identified by finding out the sets of states of the NFA for each possible input symbol



Constructing NFA for a Regular Expression

Consider a regular expression R . Apply steps 1 to 4 to construct an NFA for R inductively:

1. If R is a letter in the alphabet Σ , create a two state NFA that accepts the letter (single transition from the start state to a single final state on the letter)
2. If R is $R_1 \cdot R_2$, create an NFA by joining the two NFAs N_1 and N_2 by adding an epsilon transition from every final state of N_1 to the start state of N_2 .
3. If the R is $R_1 \mid R_2$, create an NFA by joining the two NFAs N_1 and N_2 by creating a new start state s_0 and a new final state s_f . Add an epsilon transition from s_0 the start state of R_1 and similarly for R_2 . Add an epsilon transition from every final state of N_1 to s_f and similarly for N_2 .
4. If R is R_1^* , create an NFA by adding an epsilon transition from every final state of R_1 to the start state of R_1

Alternatively, we can create a new start state s_0 with an epsilon transition to the start state of R_1 and a new final state s_f with epsilon transitions from the final states of R_1 , and then add an epsilon transition from s_f to s_0 .



Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

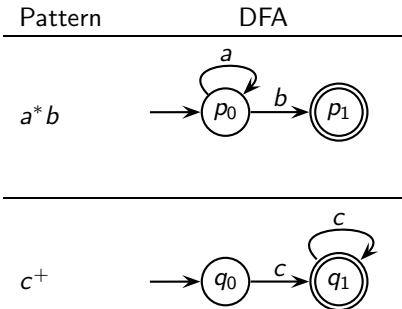
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

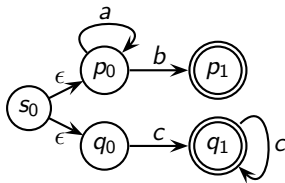
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c



Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

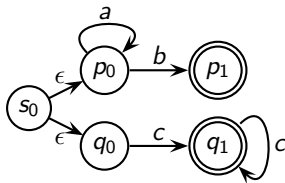
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$			

$\{s_0, p_0, q_0\}$



Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

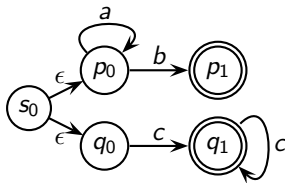
Specifying Scanners

Constructing DFAs

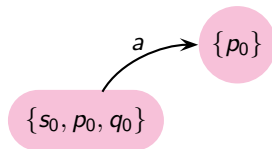
Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$	$\{p_0\}$		
$\{p_0\}$			





Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

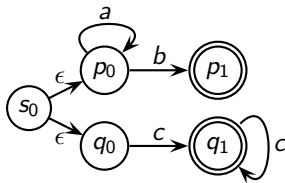
Specifying Scanners

Constructing DFAs

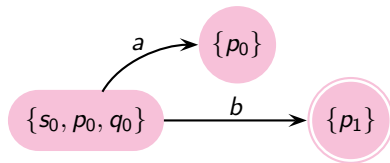
Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$	$\{p_0\}$	$\{p_1\}$	
$\{p_0\}$			
$\{p_1\}$			





Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

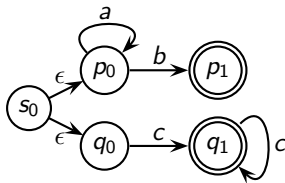
Specifying Scanners

Constructing DFAs

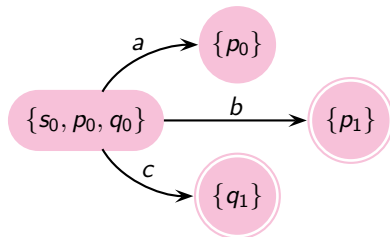
Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$	$\{p_0\}$	$\{p_1\}$	$\{q_1\}$
$\{p_0\}$			
$\{p_1\}$			
$\{q_1\}$			





Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

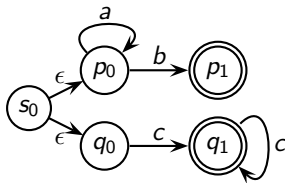
Specifying Scanners

Constructing DFAs

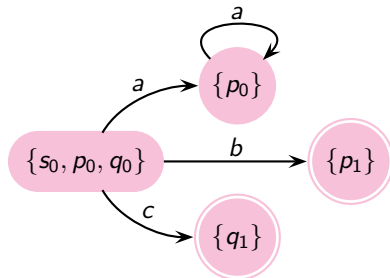
Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$	$\{p_0\}$	$\{p_1\}$	$\{q_1\}$
$\{p_0\}$	$\{p_0\}$		
$\{p_1\}$			
$\{q_1\}$			





Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

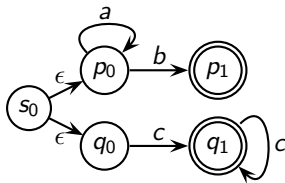
Specifying Scanners

Constructing DFAs

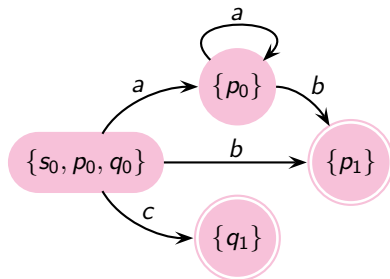
Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$	$\{p_0\}$	$\{p_1\}$	$\{q_1\}$
$\{p_0\}$	$\{p_0\}$	$\{p_1\}$	
$\{p_1\}$			
$\{q_1\}$			





Constructing DFA for Multiple Patterns: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

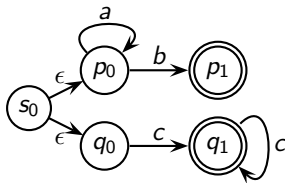
Specifying Scanners

Constructing DFAs

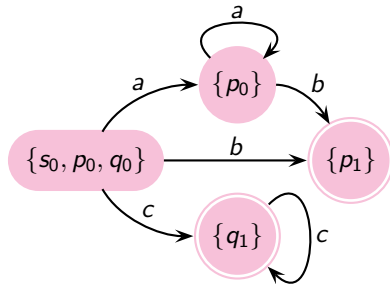
Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Transition		
	a	b	c
$\{s_0, p_0, q_0\}$	$\{p_0\}$	$\{p_1\}$	$\{q_1\}$
$\{p_0\}$	$\{p_0\}$	$\{p_1\}$	
$\{p_1\}$			
$\{q_1\}$			$\{q_1\}$





Constructing DFA for Multiple Patterns: Example 2

Let L and D denote the set of all letters and digits, respectively

Pattern	Token
int	INT
$L(L D)^*$	ID
D^+	NUM
=	=
;	;

For convenience, we will ignore the last two patterns that are completely independent

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

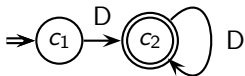
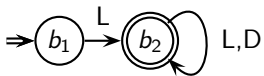
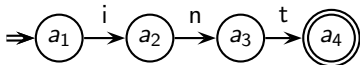
Tokenizing the Input

Representing DFAs

Minimizing DFAs



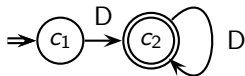
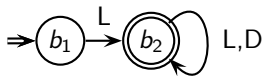
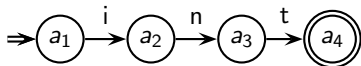
Constructing DFA for Multiple Patterns: Example 2



State	i	n	t	$L - \{i, n, t\}$	D



Constructing DFA for Multiple Patterns: Example 2



$\Rightarrow \{a_1, b_1, c_1\}$

State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

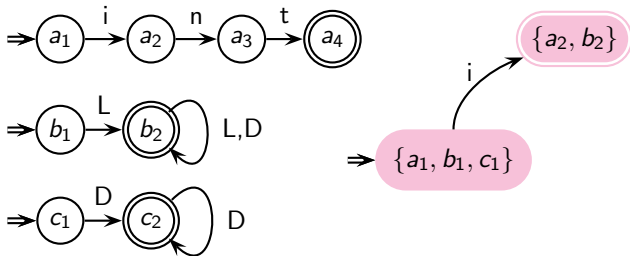
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$				
$\{a_2, b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:

Introduction

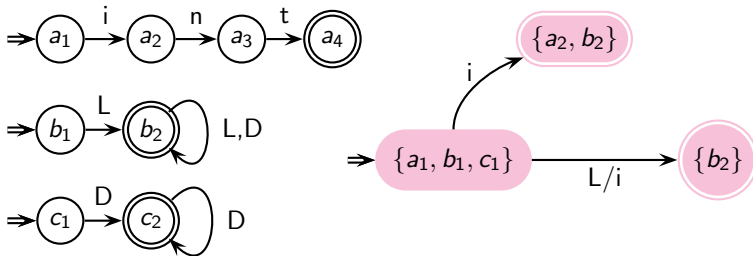
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	
$\{a_2, b_2\}$					
$\{b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:
Introduction

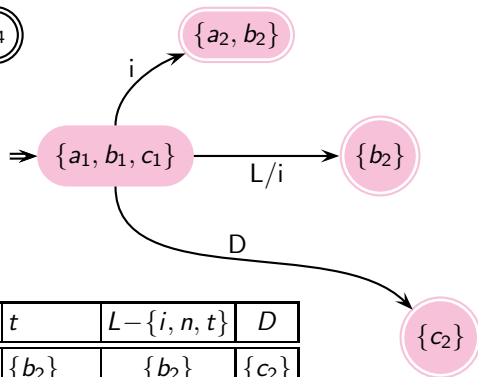
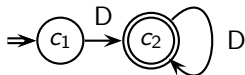
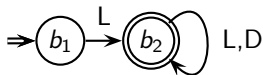
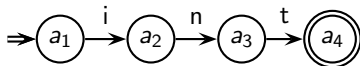
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$					
$\{b_2\}$					
$\{c_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:
Introduction

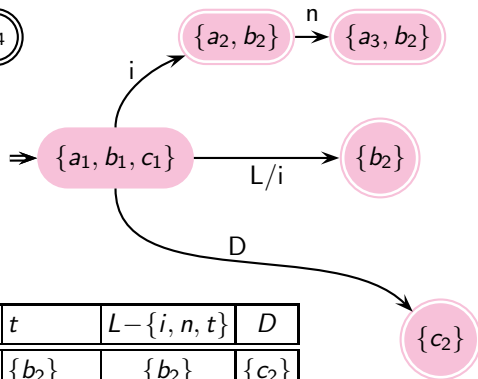
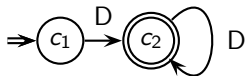
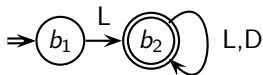
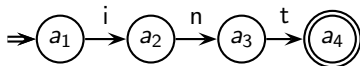
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$		$\{a_3, b_2\}$			
$\{b_2\}$					
$\{c_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:
Introduction

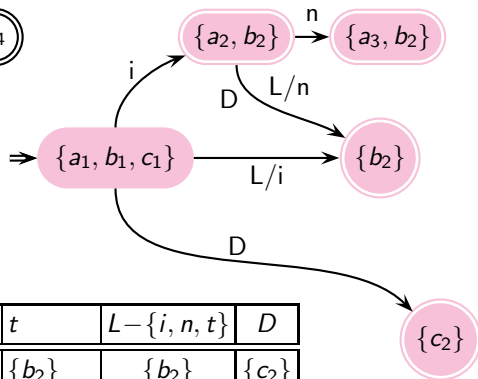
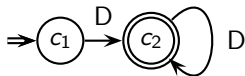
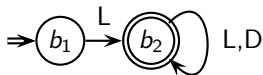
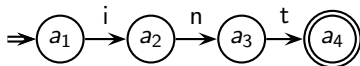
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$	$\{b_2\}$	$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{b_2\}$					
$\{c_2\}$					
$\{a_3, b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:

Introduction

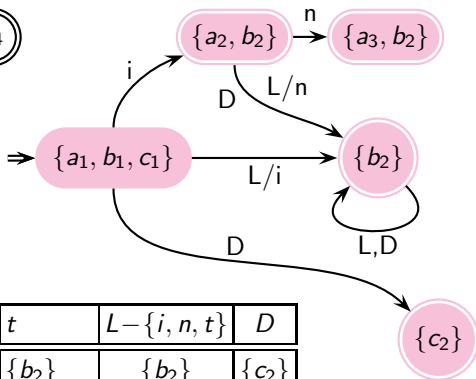
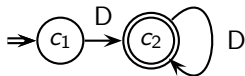
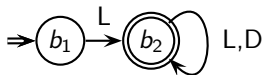
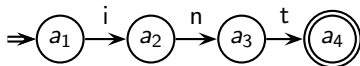
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$	$\{b_2\}$	$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{c_2\}$					
$\{a_3, b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:

Introduction

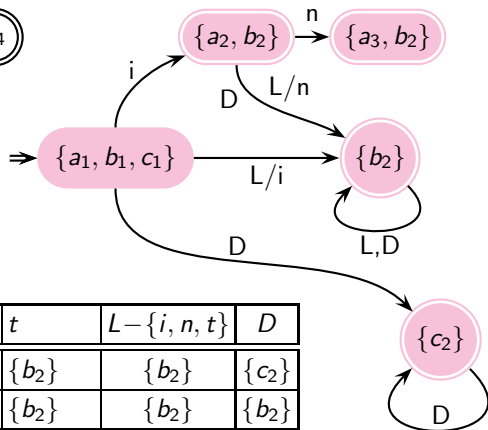
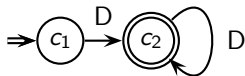
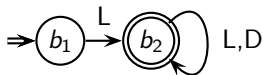
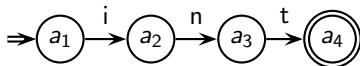
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$	$\{b_2\}$	$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{c_2\}$					$\{c_2\}$
$\{a_3, b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

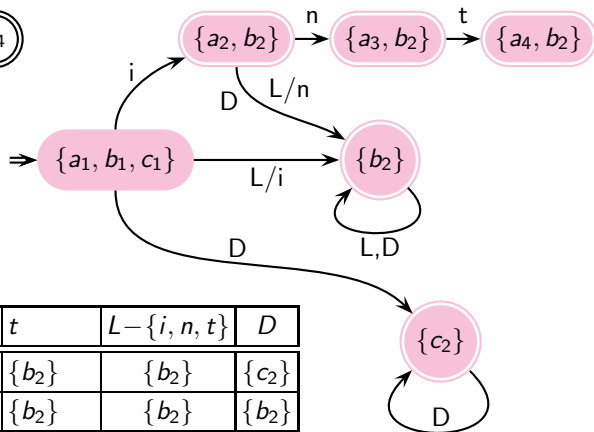
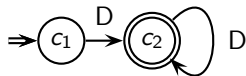
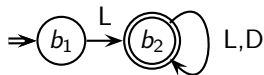
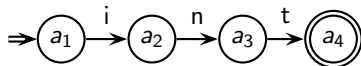
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$	$\{b_2\}$	$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{c_2\}$					$\{c_2\}$
$\{a_3, b_2\}$			$\{a_4, b_2\}$		
$\{a_4, b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

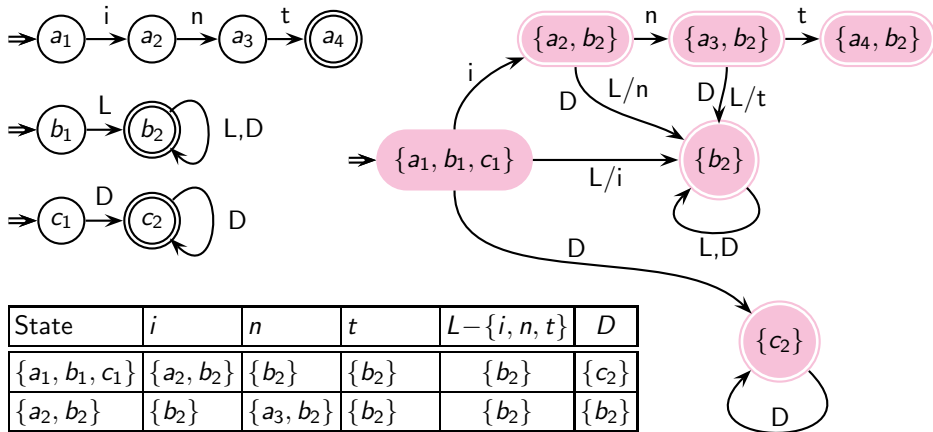
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$	$\{b_2\}$	$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{c_2\}$					$\{c_2\}$
$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{a_4, b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{a_4, b_2\}$					



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

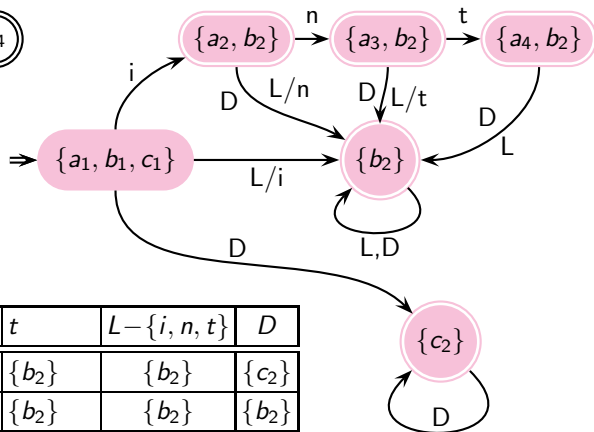
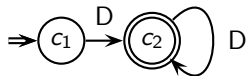
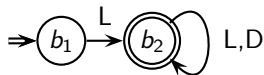
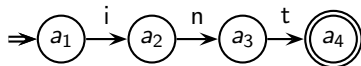
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	i	n	t	$L - \{i, n, t\}$	D
$\{a_1, b_1, c_1\}$	$\{a_2, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{c_2\}$
$\{a_2, b_2\}$	$\{b_2\}$	$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{c_2\}$					$\{c_2\}$
$\{a_3, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{a_4, b_2\}$	$\{b_2\}$	$\{b_2\}$
$\{a_4, b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$	$\{b_2\}$



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:

Introduction

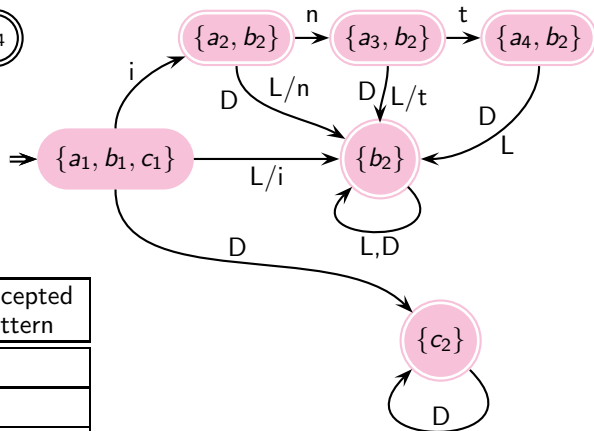
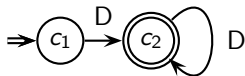
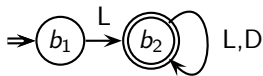
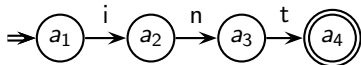
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Possible Patterns	Accepted Pattern
$\{a_1, b_1, c_1\}$		
$\{a_2, b_2\}$	ID	ID
$\{b_2\}$	ID	ID
$\{c_2\}$	NUM	NUM
$\{a_3, b_2\}$	ID	ID
$\{a_4, b_2\}$	INT, ID	INT



Constructing DFA for Multiple Patterns: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:

Introduction

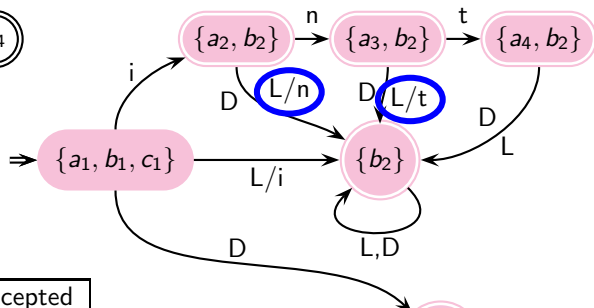
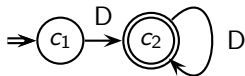
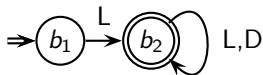
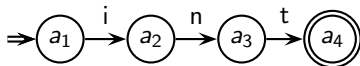
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



State	Possible Patterns	Accepted Pattern
$\{a_1, b_1, c_1\}$		
$\{a_2, b_2\}$	ID	ID
$\{b_2\}$	ID	ID
$\{c_2\}$	NUM	NUM
$\{a_3, b_2\}$	ID	ID
$\{a_4, b_2\}$	INT, ID	INT

Longest match. Lexeme "int" reaches state $\{a_4, b_2\}$ whereas lexeme "integer" reaches the state $\{b_2\}$

First matching rule preferred. Transitions L/n and L/t to state $\{b_2\}$ ensure that INT is preferred over ID for the lexeme "int"



Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

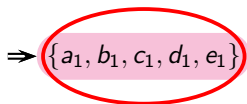
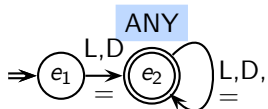
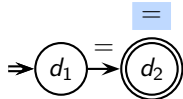
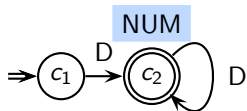
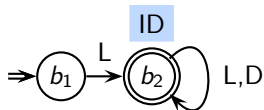
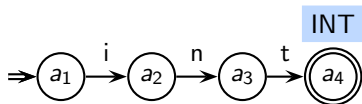
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

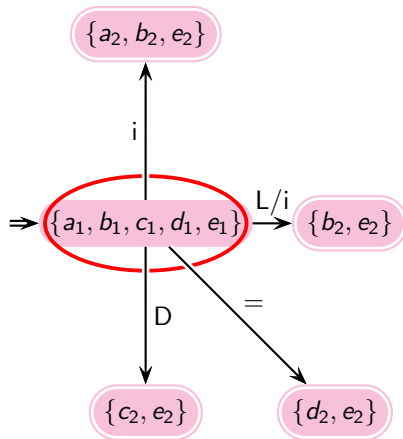
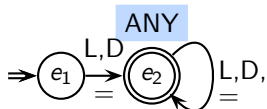
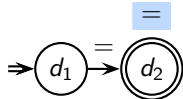
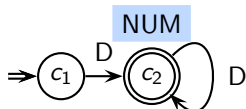
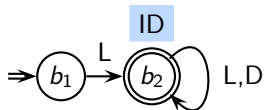
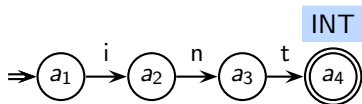
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

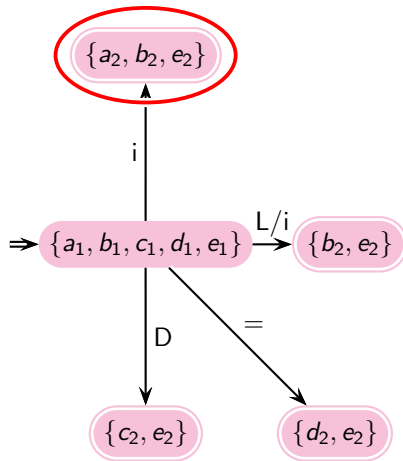
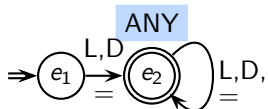
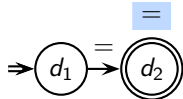
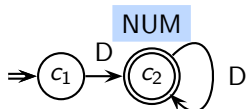
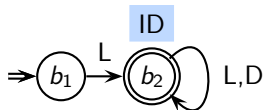
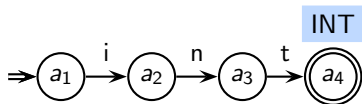
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

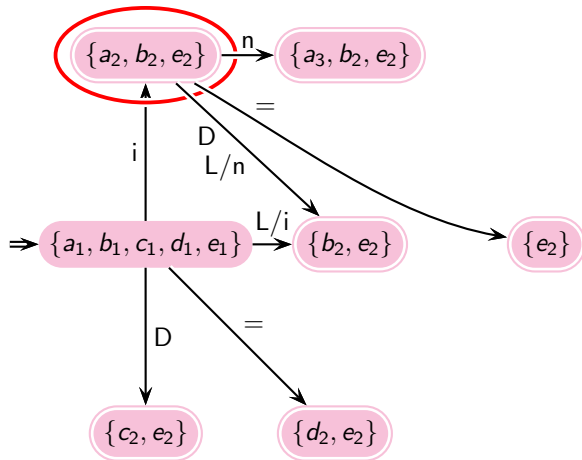
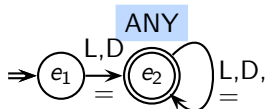
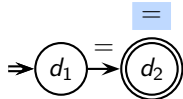
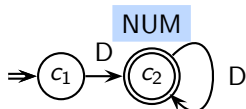
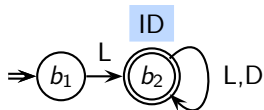
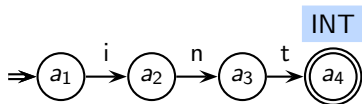
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

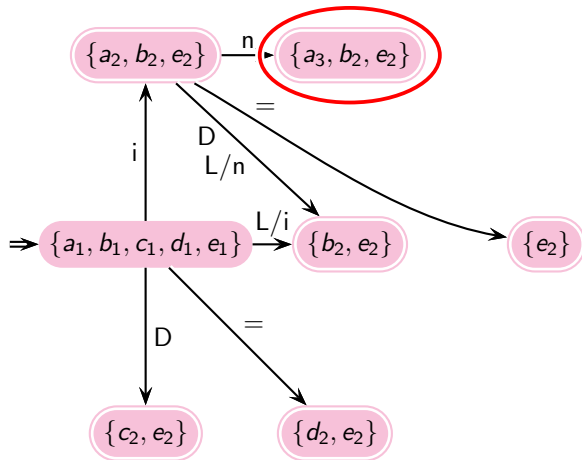
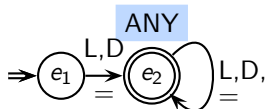
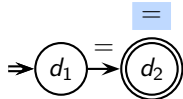
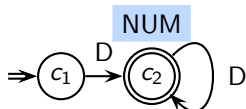
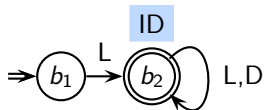
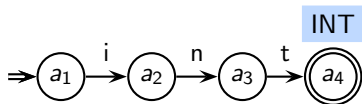
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

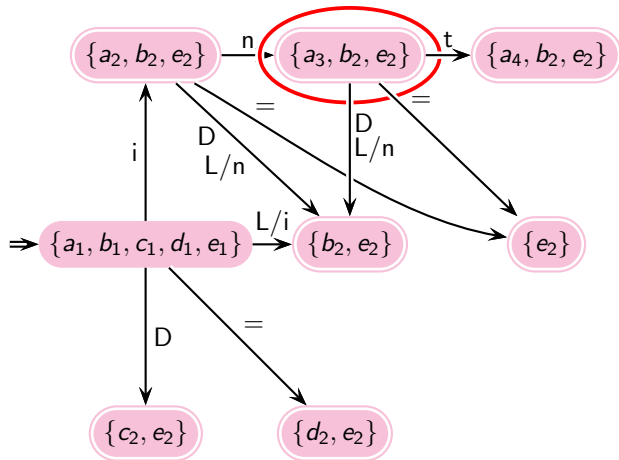
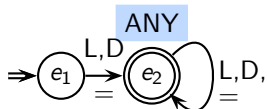
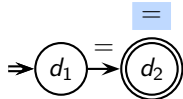
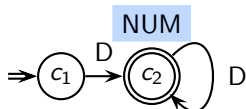
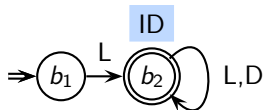
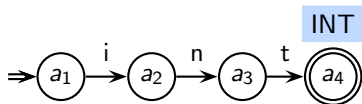
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

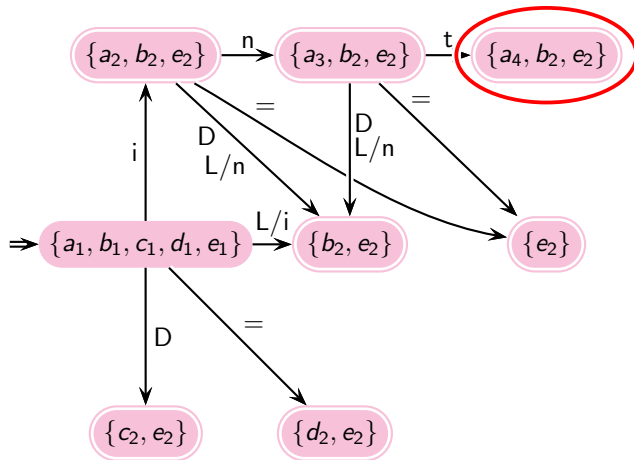
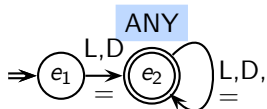
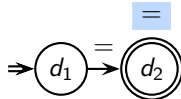
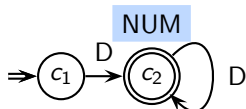
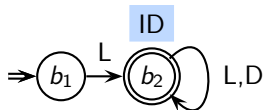
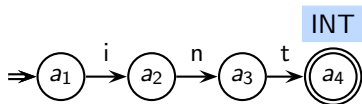
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

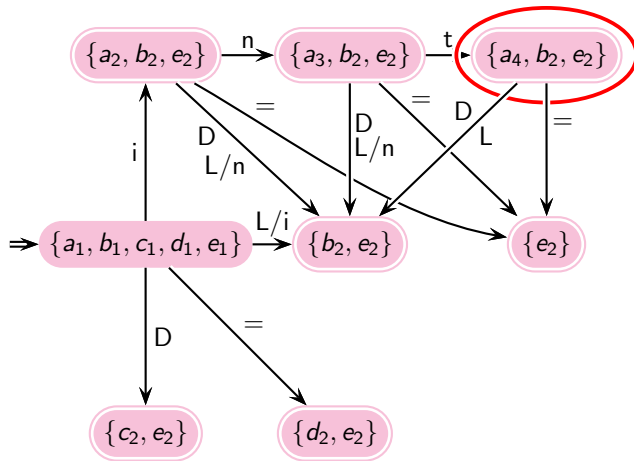
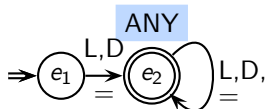
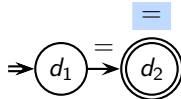
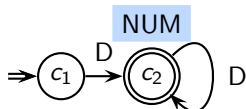
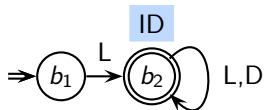
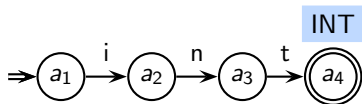
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

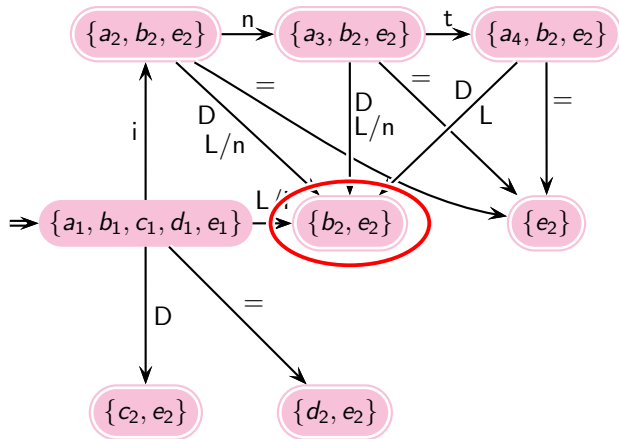
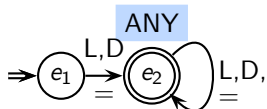
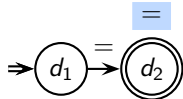
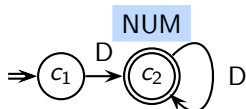
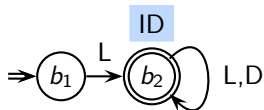
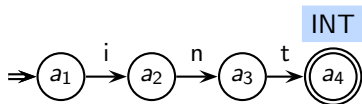
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

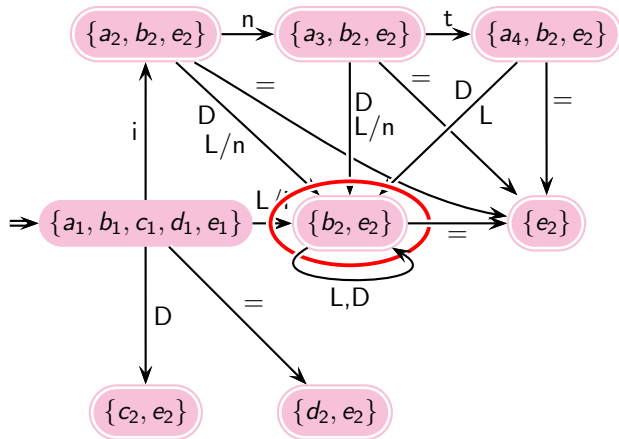
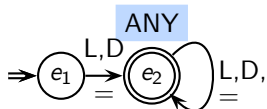
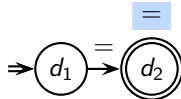
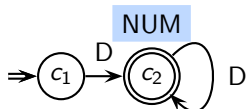
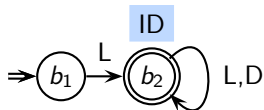
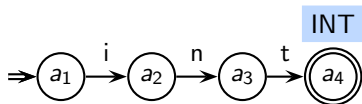
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

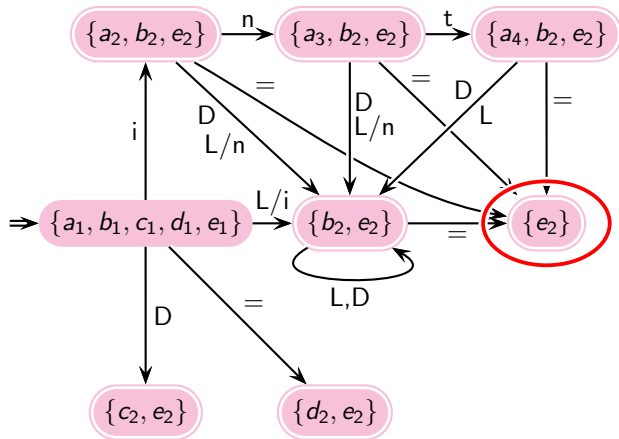
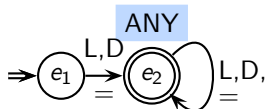
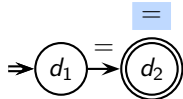
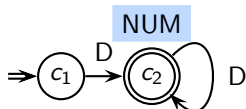
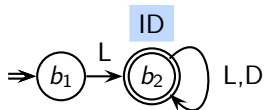
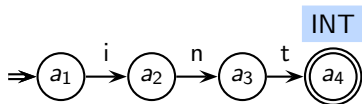
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

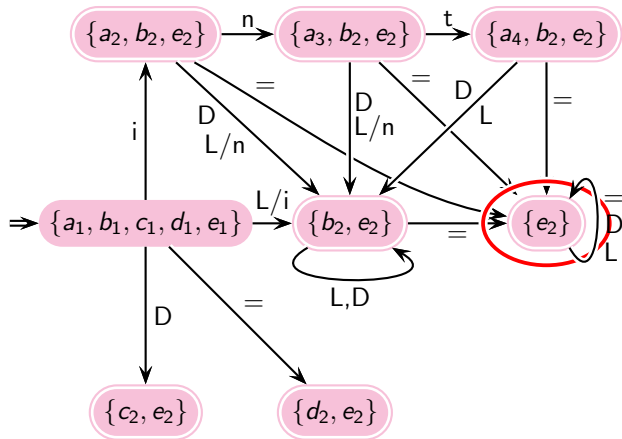
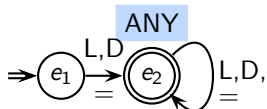
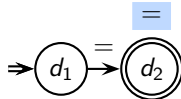
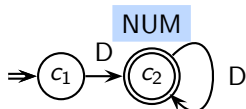
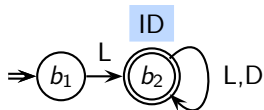
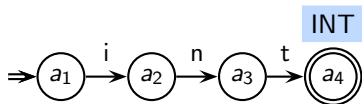
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

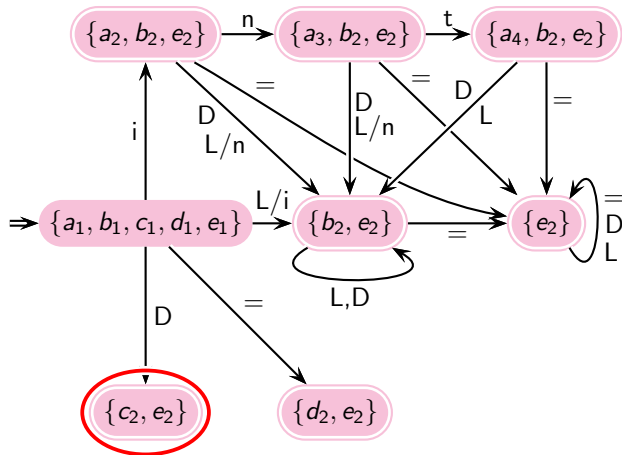
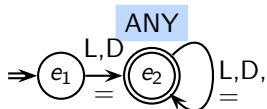
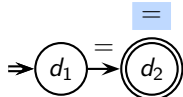
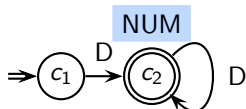
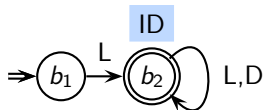
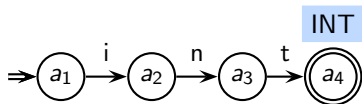
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

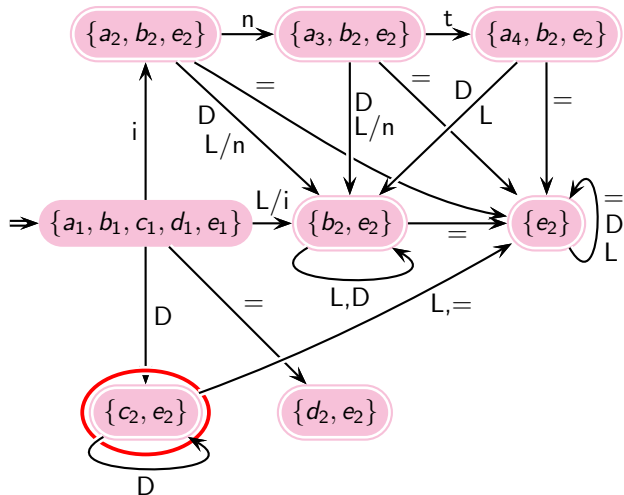
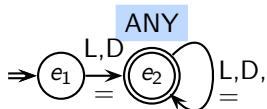
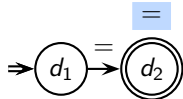
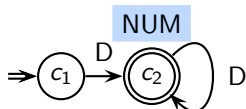
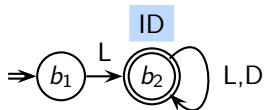
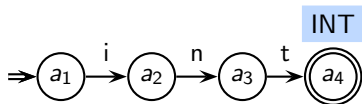
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

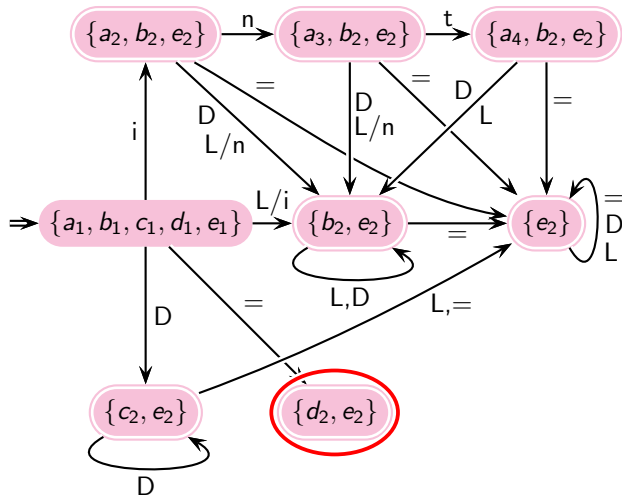
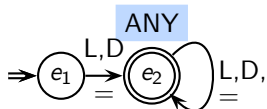
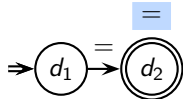
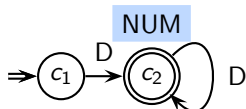
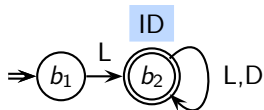
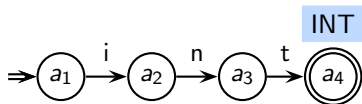
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

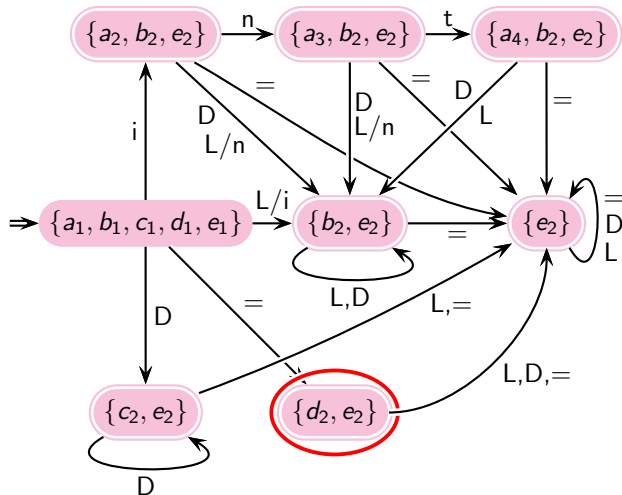
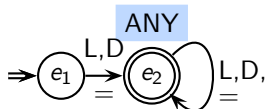
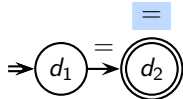
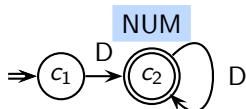
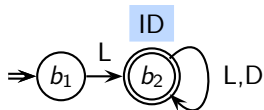
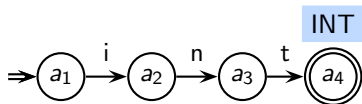
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

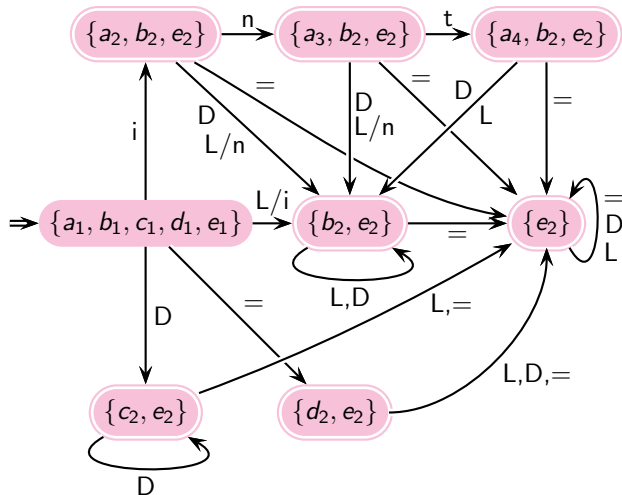
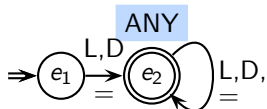
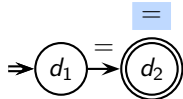
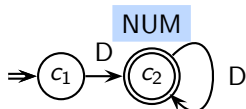
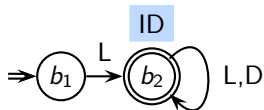
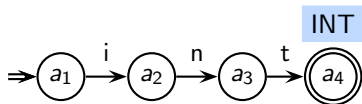
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

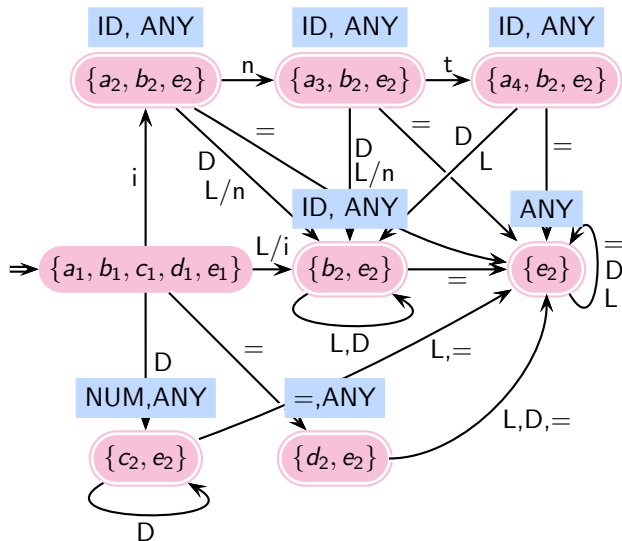
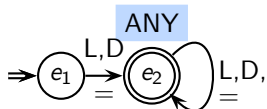
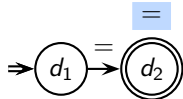
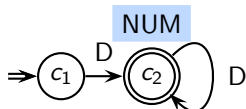
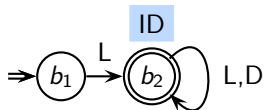
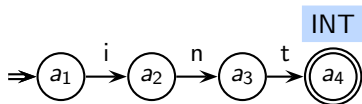
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

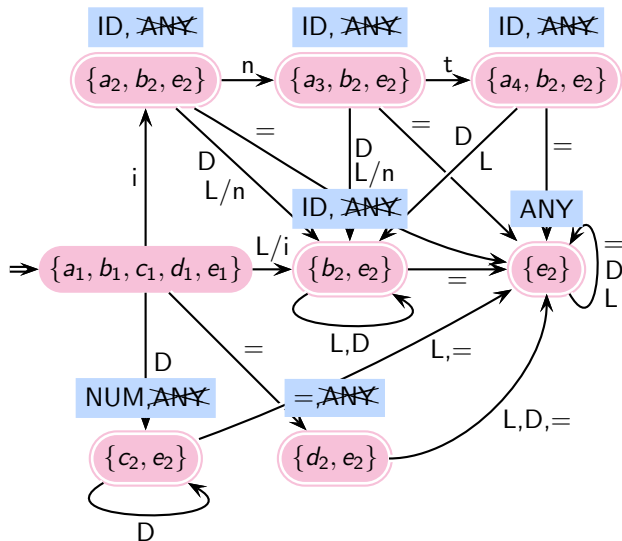
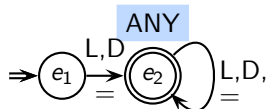
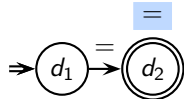
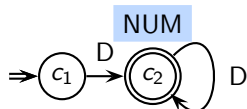
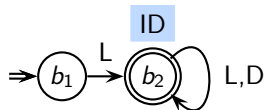
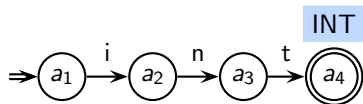
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

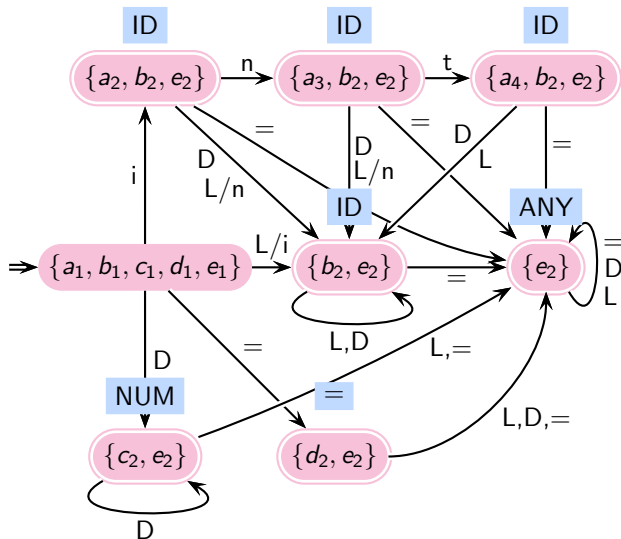
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

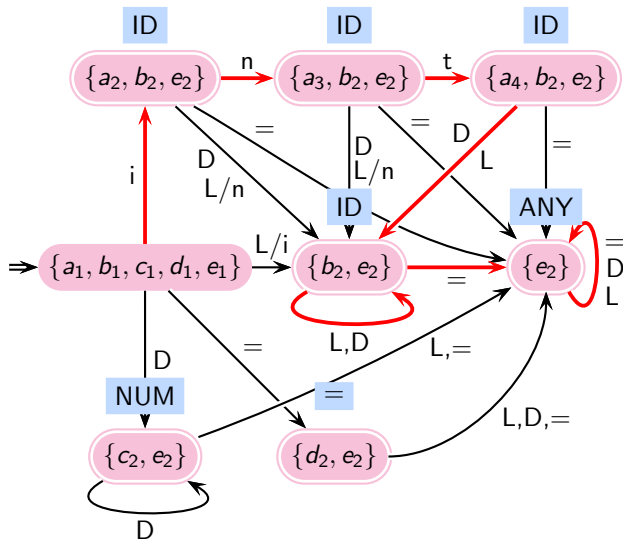




Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs



Input `int12=3` reaches state $\{e_2\}$ along the red transitions, recognizing the token ANY

Hence the "." pattern should be used with caution in a lex script

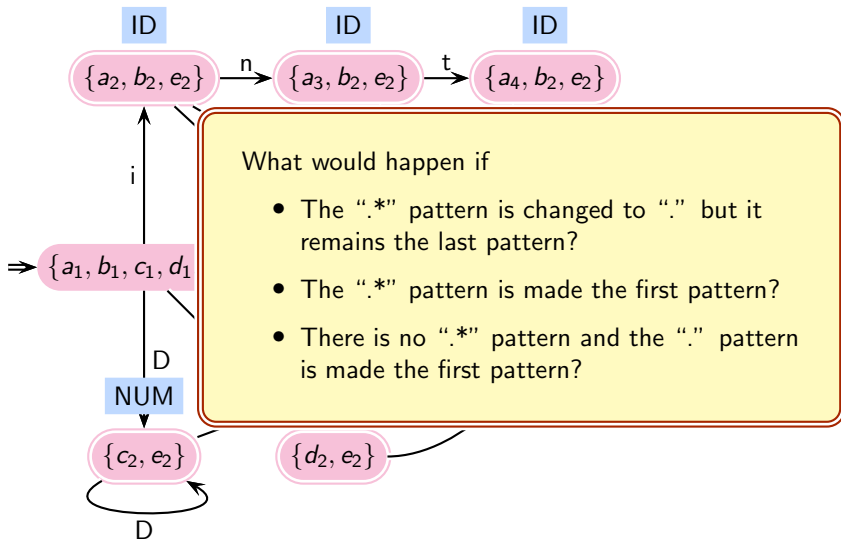
The "." pattern is much safer in a lex script



Constructing DFA for Multiple Patterns: Example 3

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Tokenizing the Input Using DFAs



An Example for Scanning: Specifications

Let L and D denote the set of all letters and digits, respectively

Pattern	Token
int	INT
$L(L D)^*$	ID
D^+	NUM
=	=
;	;

We will scan the input string int int32=5;↵

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Example for Scanning: DFA for the Patterns



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Formally, a Deterministic Finite Automaton (DFA) is a five tuple

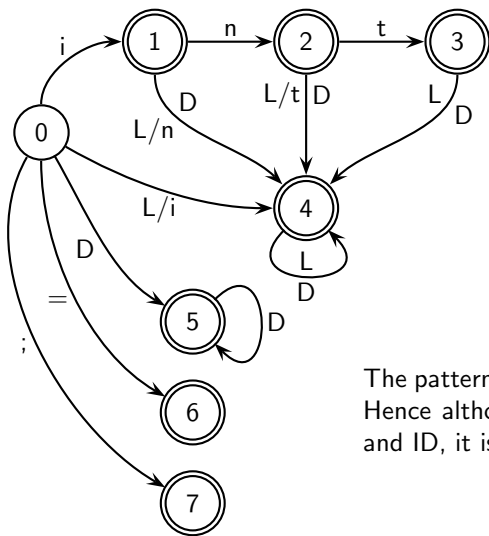
$$(\Sigma, S, s_0, \delta, F)$$

where

- Σ is the input alphabet
- S is the set of states
- $s_0 \in S$ is a unique start state
- $\delta : S \times \Sigma \rightarrow S$ is a transition function
- $F \subseteq S$ is a set of final states



Example for Scanning: DFA for the Patterns



States	Action
3	Found INT
1, 2, 4	Found ID
5	Found NUM
6	Found =
7	Found ;

The patterns for INT precedes the pattern for ID
Hence although state 3 could accept both INT and ID, it is made to accept only INT



A Format to Show A Trace of Scanning

Step No	State	MatchedString	Buffer	NextChar	LastFinalState	MarkedPos	Action
---------	-------	---------------	--------	----------	----------------	-----------	--------

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

- **State (S).** Current State
- **MatchedString (MS).** Prefix of the buffer matched to identify a lexeme
- **Buffer.**
- **NextChar (NC).** The next character in the input; it will be shifted to the buffer if there is a valid transition in the DFA
- **LastFinalState (LFS).** The last final state seen
- **MarkedPos (MP).** The position of the character (in the buffer) just after the last seen lexeme
- **Action.**



A Format to Show A Trace of Scanning

Step No	State	MatchedString	Buffer	NextChar	LastFinalState	MarkedPos	Action
---------	-------	---------------	--------	----------	----------------	-----------	--------

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning

Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

- **State (S).** Current State

- **M**

When there is no transition on Nextchar,

- **B**

- if MarkedPos is -1, no final state is seen, the first character in the buffer is discarded, and the second character becomes NextChar,

- **N**

- **t**

- **L**

- otherwise, the lexeme upto MarkedPos (excluding it) is returned, the character at MarkedPos becomes NextChar

- **M**

- **s**

In either case, the LastFinalState is set to -1 and the state is set to 0

- **Action.**

if

st



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

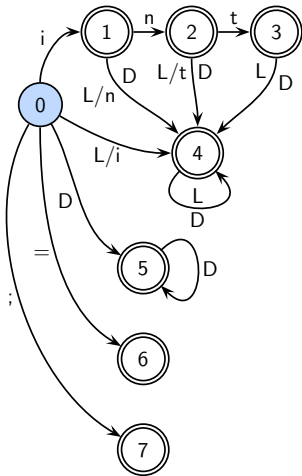
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

i n t _ i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

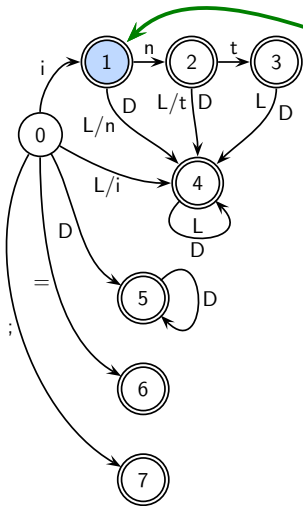
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input "int int32=5;↵"



Last Final State

Matched String

i n t _ i n t 3 2 = 5 ; ↵

Marked Position

Next Char



Scanning the Input “int int32=5;↵”

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

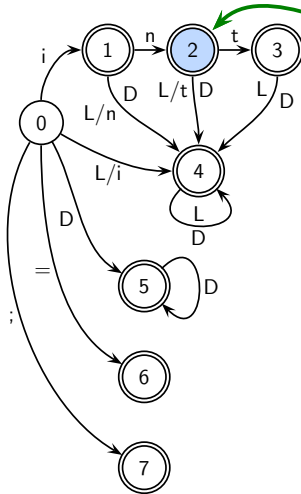
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Last Final State

Matched String

i n t _ i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

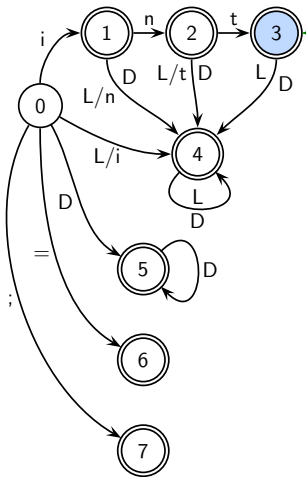
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Found INT

int ↵ int32 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

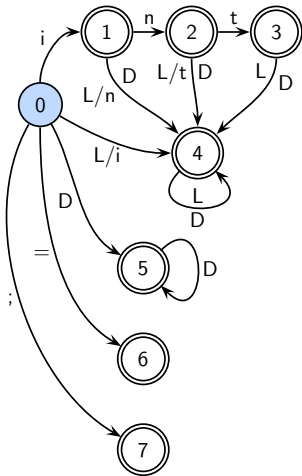
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Discard

↵ i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

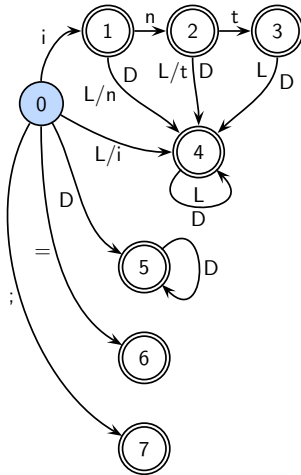
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

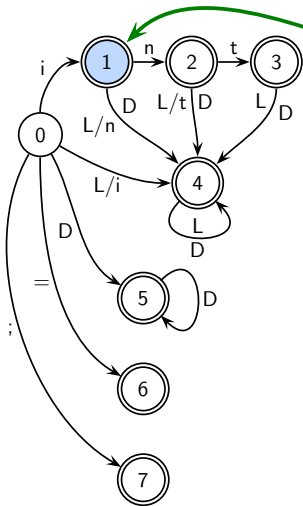
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

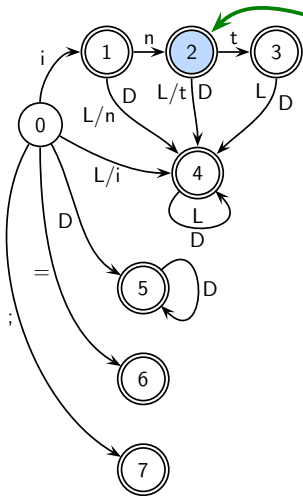
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

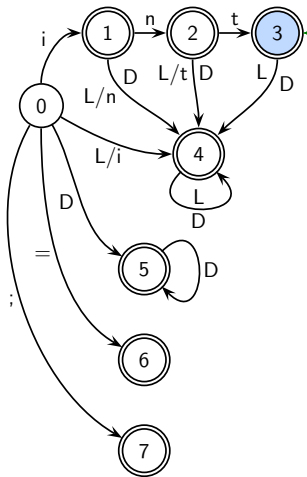
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



Scanning the Input "int32=5;↵"

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

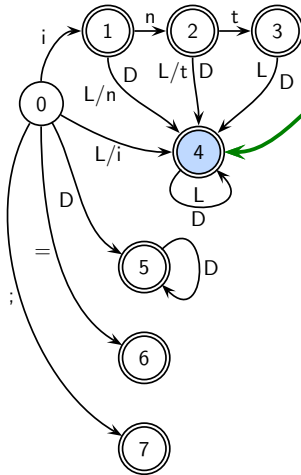
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Last Final State

Matched String

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

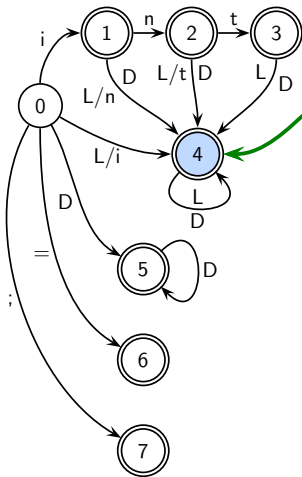
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Found ID

Matched String

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

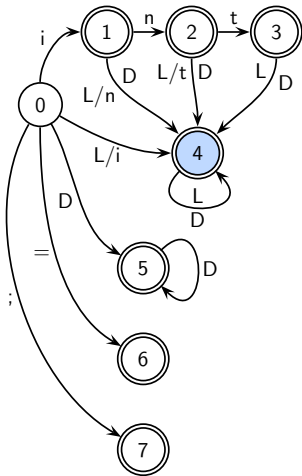
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

These patterns do not illustrate the situation when Marked Position is different from Next Char. The next example demonstrates it.

i n t 3 2 = 5 ; ↵

Marked Position

Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

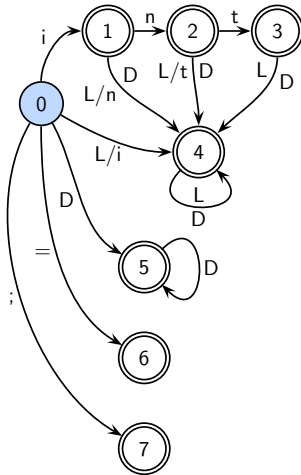
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

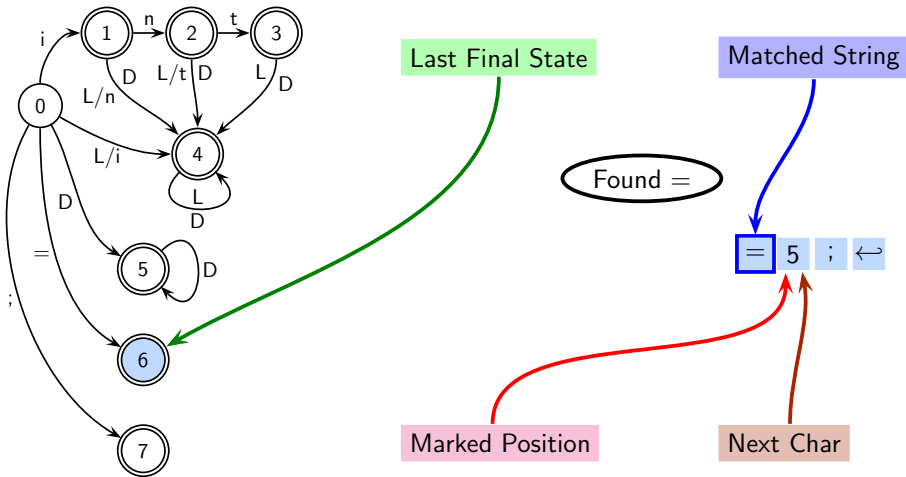
= 5 ; ↵

Marked Position

Next Char



Scanning the Input "int int32=5;↵"





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

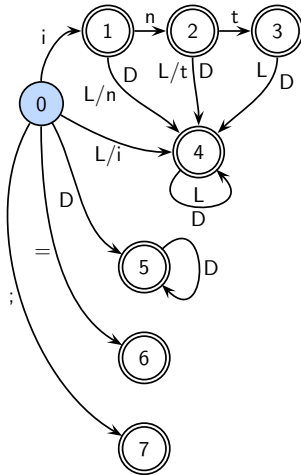
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

5 ; ↵

Marked Position

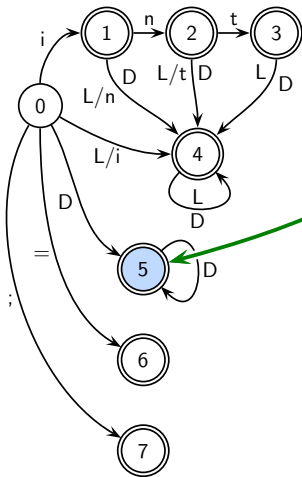
Next Char



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Found NUM

Marked Position

Next Char

5 ; ↵



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

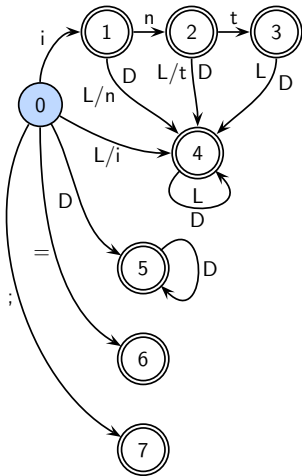
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Marked Position

Next Char

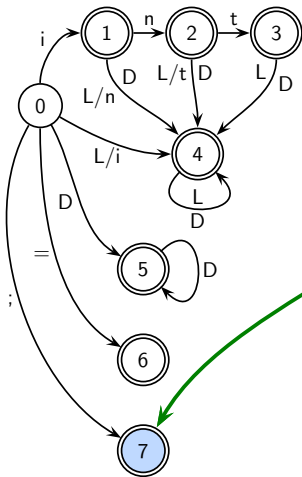
; ↵



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Found ;

Marked Position

Next Char

;



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

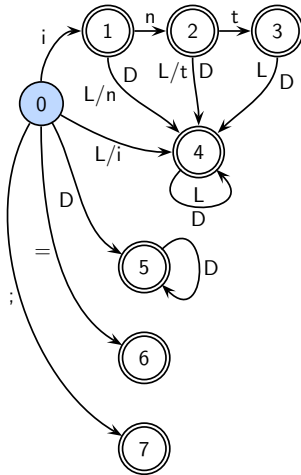
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Discard

Marked Position

Next Char

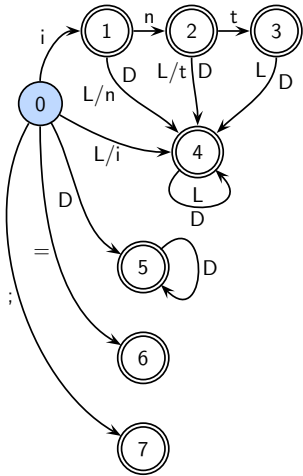




IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Scanning the Input “int int32=5;↵”



Last Final State

Matched String

Marked Position

Next Char



The Trace of Scanning the Input "int int32=5;↵"

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

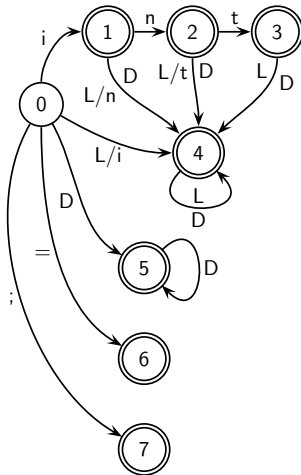
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



SNo	S	MS	Buffer	NC	LFS	MP	Action
1	0		int↵int32=5;↵	i			
2	1	i	int↵int32=5;↵	n	1	1	
3	2	in	int↵int32=5;↵	t	2	2	
4	3	int	int↵int32=5;↵	↵	3	3	Found INT
5	0		↵int32=5;↵	↵			Discard ↵
6	0		int32=5;↵	i			
7	1	i	int32=5;↵	n	1	1	
8	2	in	int32=5;↵	t	2	2	
9	3	int	int32=5;↵	3	3	3	
10	4	int3	int32=5;↵	2	4	4	
11	4	int32	int32=5;↵	=	4	5	Found ID
12	0		=5;↵	=			
13	6	=	=5;↵	5	6	1	Found =
14	0		5;↵	5			
15	5	5	5;↵	;	5	1	Found NUM
16	0		;↵	;			
17	7	;	↵	↵	7		Found ;

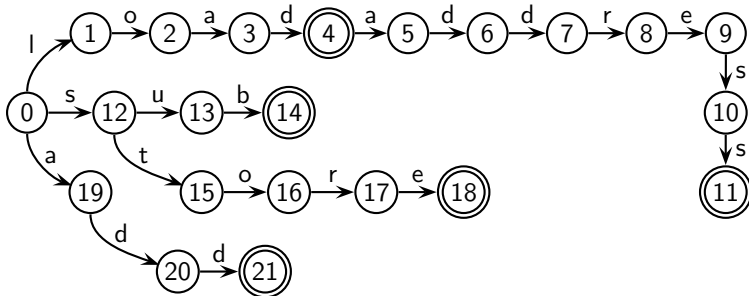


Tutorial Problem On Scanning

- Find the occurrences of following substrings in a given input string

load, loadaddress, add, sub, store

- Use the following automata

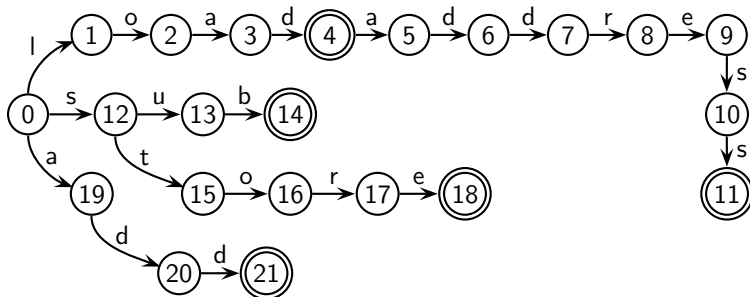


- Scan two input strings `loadsubadd↵` and `loadaddsub↵`



The Role of MarkedPos

Observe the role of MarkedPos for the input `loadaddsub`↵



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

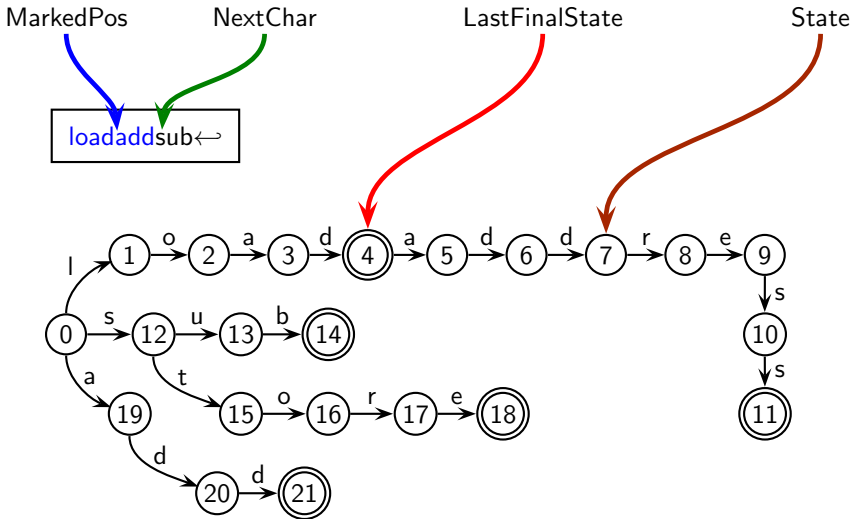
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

The Role of MarkedPos





The Role of MarkedPos

MarkedPos

-1

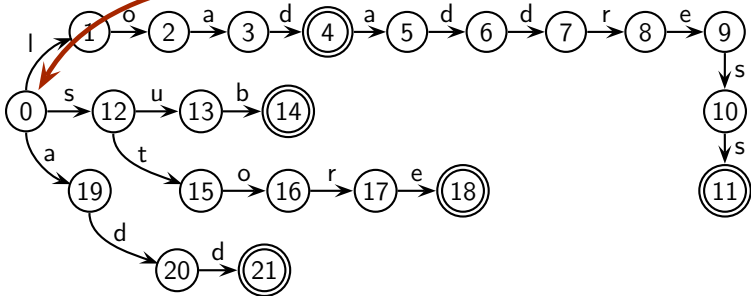
NextChar

LastFinalState

-1

State

loadaddsub↔



Demo of Scan Trace



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

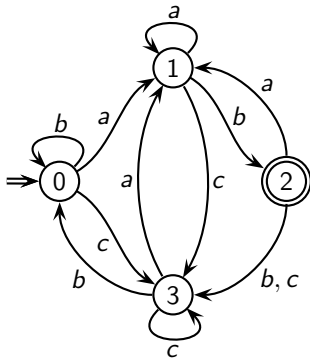
Representing DFAs

Minimizing DFAs

Representing DFAs Using Four Arrays



DFA to be Represented Using Four Arrays: Example 1



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



DFA to be Represented Using Four Arrays: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

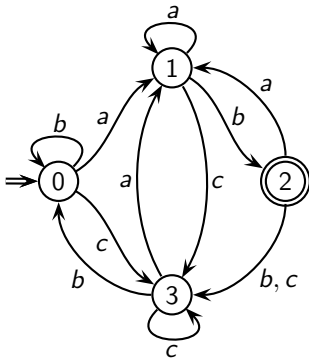
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3



DFA to be Represented Using Four Arrays: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

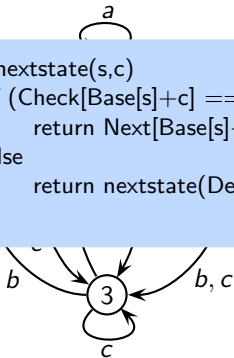
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
  return Next[Base[s]+c];
  else
    return nextstate(Default[s],c);
}
```



States 0 and 3 have identical transitions. Transitions in states 1 and 2 differ from them only on b.

Char	Code
a	0
b	1
c	2

	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1		
2		
3		

	Next	Check
0		
1		
2		
3		
4		
5		

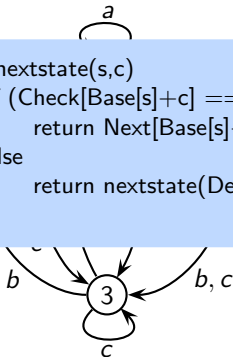


DFA to be Represented Using Four Arrays: Example 1

We choose to fill the entries for state 0 first (state 3 could also have been used)

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
  return Next[Base[s]+c];
  else
    return nextstate(Default[s],c);
}
```

Char	Code
a	0
b	1
c	2



	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1		
2		
3		

	Next	Check
0		
1		
2		
3		
4		
5		



DFA to be Represented Using Four Arrays: Example 1

The Check array contains 0 to confirm that the corresponding entries in the next array are for state 0

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
  return Next[Base[s]+c];
  else
  return nextstate(Default[s],c);
}
```

Char	Code
a	0
b	1
c	2



	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1		
2		
3		

	Next	Check
0	1	0
1	0	0
2	3	0
3		
4		
5		

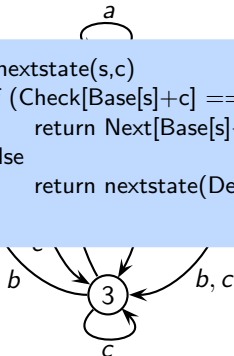


DFA to be Represented Using Four Arrays: Example 1

The Check array contains 0 to confirm that the corresponding entries in the next array are for state 0

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
  return Next[Base[s]+c];
  else
  return nextstate(Default[s],c);
}
```

Char	Code
a	0
b	1
c	2



	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1		
2		
3		

	Next	Check
0	1	0
1	0	0
2	3	0
3		
4		
5		



DFA to be Represented Using Four Arrays: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

```

int nextstate(s,c)
{ if (Check[Base[s]+c] == -1)
    return Next[Base[s]+c];
  else
    return nextstate(Default[s],c);
}

```

For state 1, we reuse the transitions on a and c from state 0 but need to enter transition on b explicitly. We do this using the next free entry (index 3) in the next array and back calculating the base of state 1.

Char	Code
a	0
b	1
c	2



	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1	0	
2		
3		

	Next	Check
0	1	0
1	0	0
2	3	0
3	2	1
4		
5		



DFA to be Represented Using Four Arrays: Example 1

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
    return Next[Base[s]+c];
  else
    return nextstate(Default[s],c);
}
```

The variation in state 2 is similar to that for state 1. We reuse the transitions on a and c from state 0 but enter transition on b explicitly in the next free entry (index 4) in the next array and back-calculate the base of state 2.



Char	Code
a	0
b	1
c	2

	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1	0	
2	0	
3		

	Next	Check
0	1	0
1	0	0
2	3	0
3	2	1
4	3	2
5		



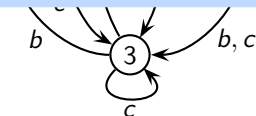
DFA to be Represented Using Four Arrays: Example 1

State 3 is identical to state 0. We have shown here its base as same as for state 0.

(In practice, lex begins the entries from index 1 and keeps index 0 free for such entries. We have ignored it because it is a matter of details.)

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
  return Next[Base[s]+c];
  else
  return nextstate(Default[s],c);
}
```

Char	Code
a	0
b	1
c	2



	a	b	c
0	1	0	3
1	1	2	3
2	1	3	3
3	1	0	3

State	Default	Base
0		
1	0	
2	0	
3	0	

	Next	Check
0	1	0
1	0	0
2	3	0
3	2	1
4	3	2
5		

The Intuition Behind Four Array Representation



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

How to find the appropriate space in Next array for a state?

- View the entries (in the row of the state) that are required to be stored as “pins” separated by the entries that are not required to be stored
- View the positions in the Next array that do not contain a transition as “holes”
- Try to match the pattern (i.e. separation) of pins with that of the available holes



DFA to be Represented Using Four Arrays: Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

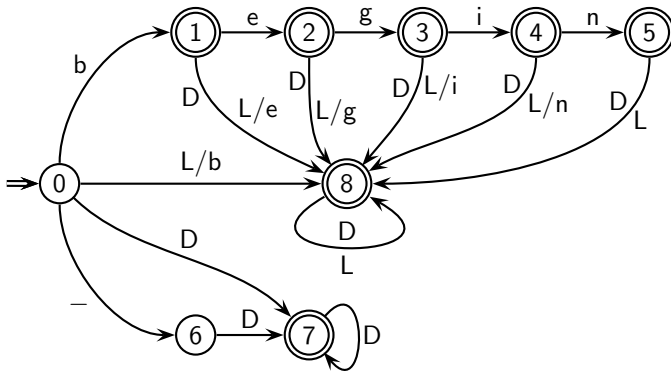
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Set	Characters
L	a to z
D	0 to 9

Pattern	Token
begin	BEGIN
$L(L D)^*$	ID
$(- \epsilon)D^+$	NUM



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Table Representation for Example 2

In the following, L denotes any letter from a to z other than b, e, g, i, n because these letters are listed separately

	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

Character	Code
a - z	0 - 25
0 - 9	26 - 35
-	36



Choice of Default States for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

- States 8 and 6 are represented independently
- State 6 is the default state for state 7
- State 8 is the default state for all other states



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Four Arrays Representation for Example 2

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0		
1		
2		
3		
4		
5		
6		
7		
8		

	N	C
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
17		
18		
19		

	N	C
20		
21		
22		
23		
24		
25		
26		
27		
28		
29		
30		
31		
32		
33		
34		
35		
36		
37		
38		
39		

	N	C
40		
41		
42		
43		
44		
45		
46		
47		
48		
49		
50		
51		
52		
53		
54		
55		
56		
57		
58		
59		

	N	C
60		
61		
62		
63		
64		
65		
66		
67		
68		
69		
70		
71		
72		
73		
74		
75		
76		
77		
78		
79		



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Four Arrays Representation for Example 2

			N C		N C		N C		N C	
S: State	S	D B	0	8 8	20	8 8	40		60	
D: Default			1	8 8	21	8 8	41		61	
B: Base			2	8 8	22	8 8	42		62	
N: Next			3	8 8	23	8 8	43		63	
C: Check			4	8 8	24	8 8	44		64	
			5	8 8	25	8 8	45		65	
			6	8 8	26	8 8	46		66	
			7	8 8	27	8 8	47		67	
			8	8 8	28	8 8	48		68	
	9		9	8 8	29	8 8	49		69	
	10		10	8 8	30	8 8	50		70	
	11		11	8 8	31	8 8	51		71	
	12		12	8 8	32	8 8	52		72	
	13		13	8 8	33	8 8	53		73	
	14		14	8 8	34	8 8	54		74	
	15		15	8 8	35	8 8	55		75	
	16		16	8 8	36		56		76	
	17		17	8 8	37		57		77	
	18		18	8 8	38		58		78	
	19		19	8 8	39		59		79	



Four Arrays Representation for Example 2

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0	8	●
1		
2		
3		
4		
5		
6		
7		
8		●

N	C
0	8
1	8
2	8
3	8
4	8
5	8
6	8
7	8
8	8
9	8
10	8
11	8
12	8
13	8
14	8
15	8
16	8
17	8
18	8
19	8

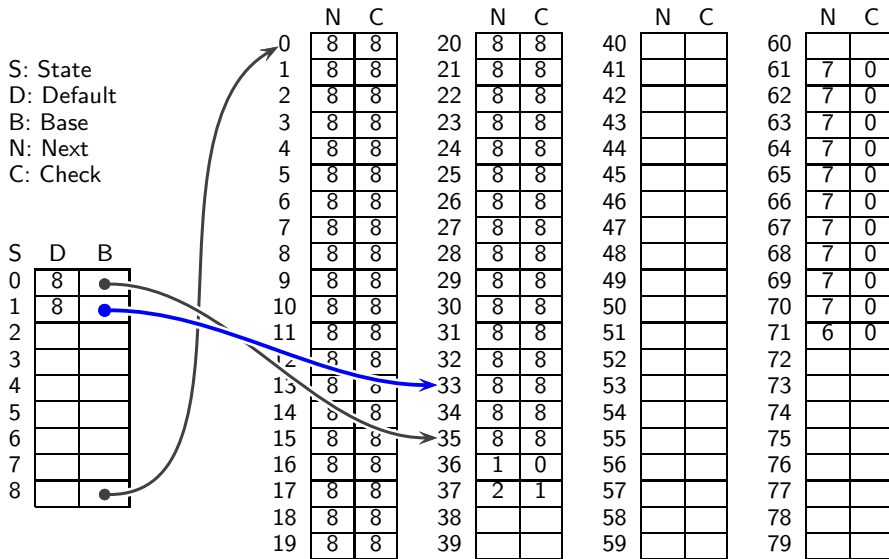
N	C
20	8
21	8
22	8
23	8
24	8
25	8
26	8
27	8
28	8
29	8
30	8
31	8
32	8
33	8
34	8
35	8
36	1
37	
38	
39	

N	C
40	
41	
42	
43	
44	
45	
46	
47	
48	
49	
50	
51	
52	
53	
54	
55	
56	
57	
58	
59	

N	C
60	
61	7
62	7
63	7
64	7
65	7
66	7
67	7
68	7
69	7
70	7
71	6
72	
73	
74	
75	
76	
77	
78	
79	



Four Arrays Representation for Example 2





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

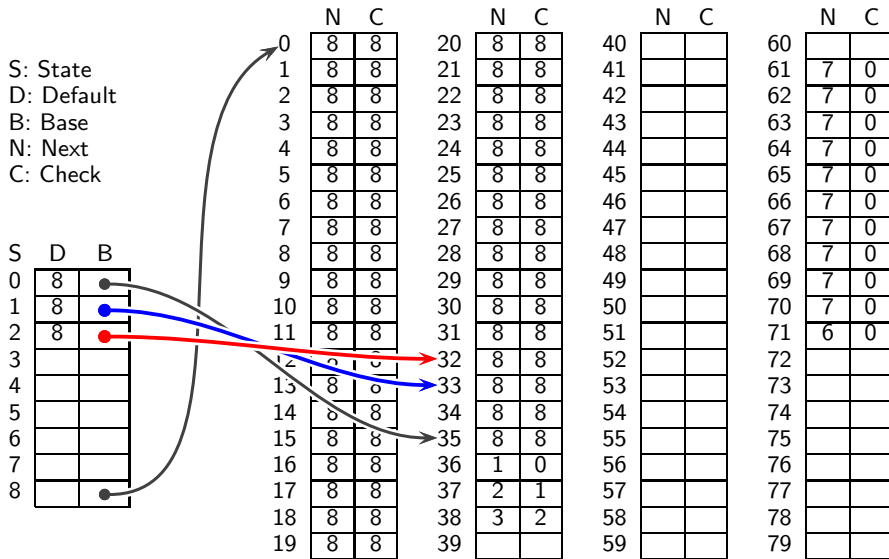
Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Four Arrays Representation for Example 2

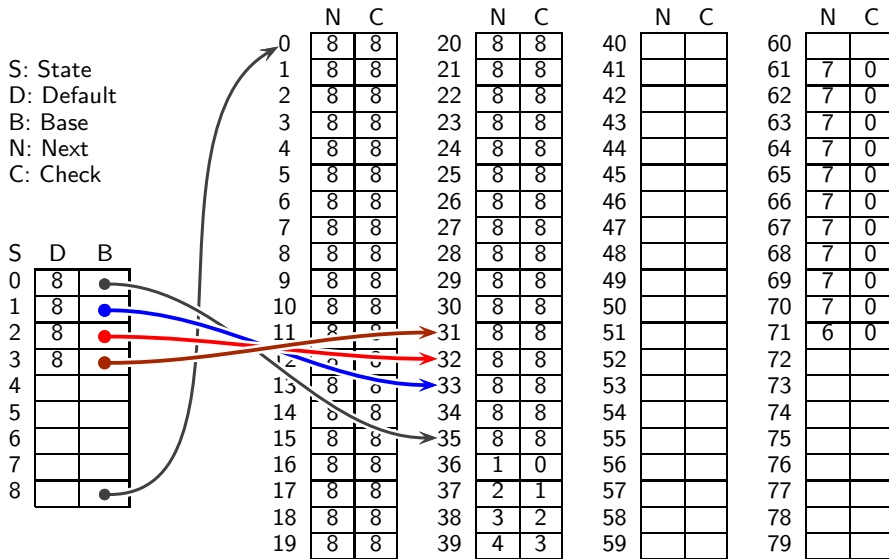




IIT Bombay
cs302: Implementation
of Programming
Languages

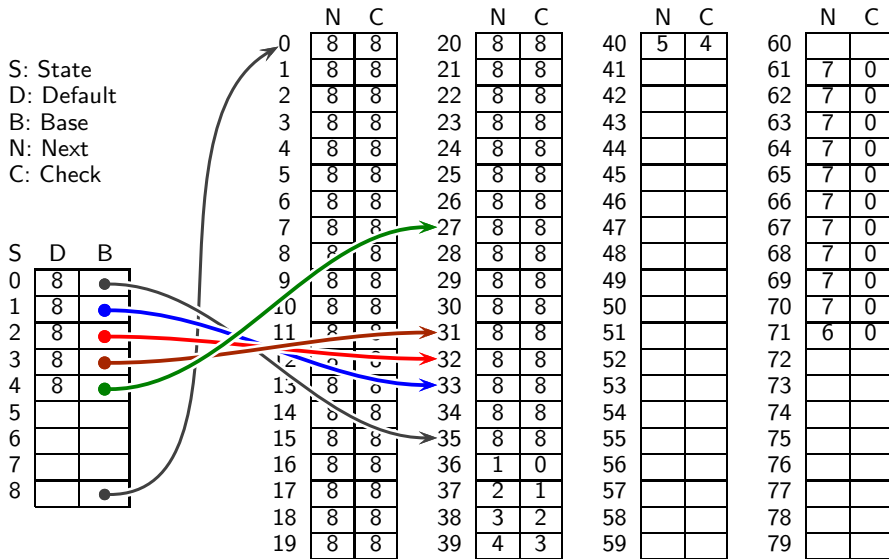
Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

Four Arrays Representation for Example 2



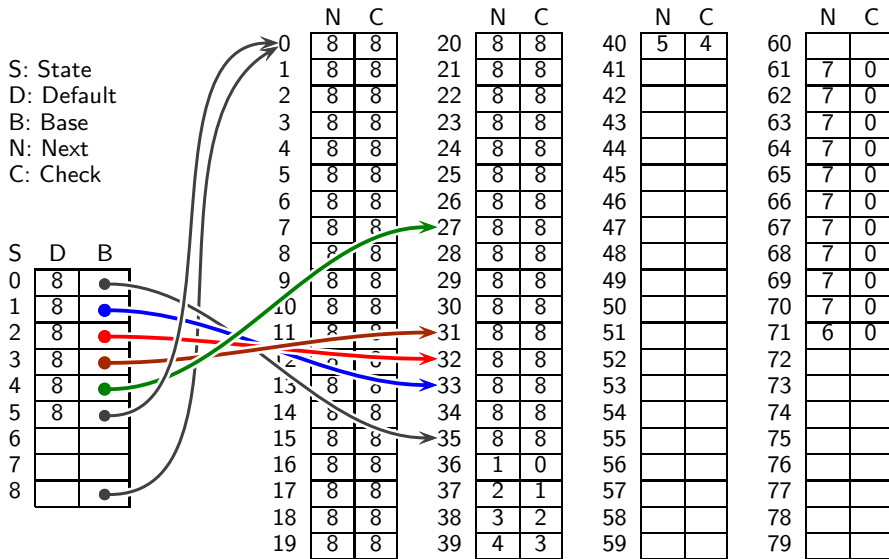


Four Arrays Representation for Example 2





Four Arrays Representation for Example 2





Four Arrays Representation for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

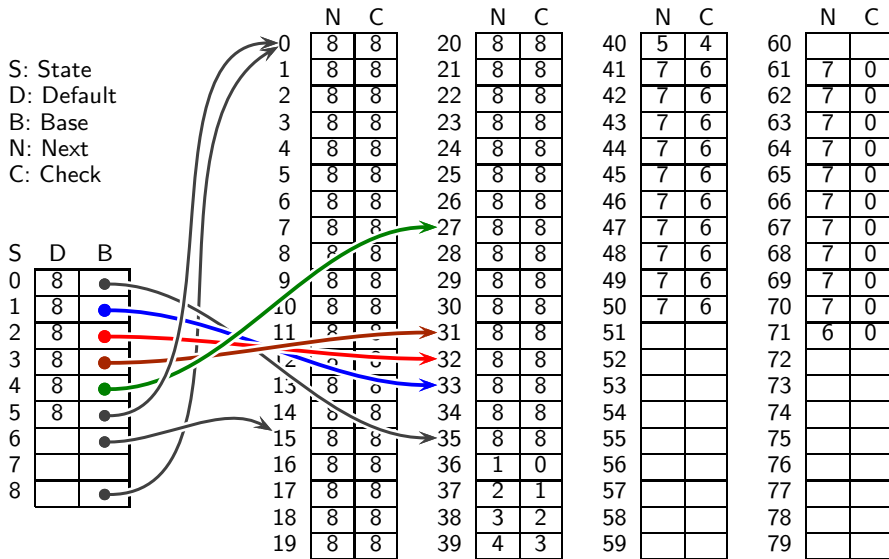
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Four Arrays Representation for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

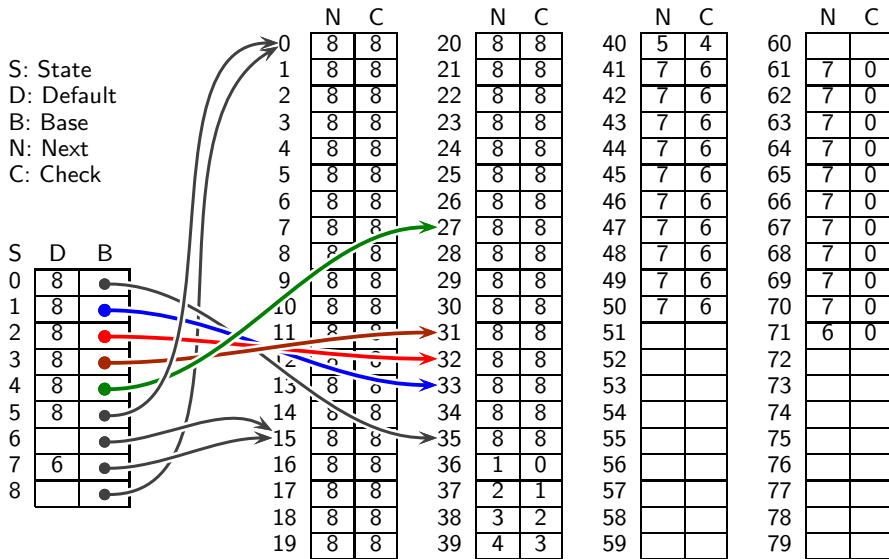
Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs





Size Comparison for Example 2

- Space for a 2 dimensional table

$$\text{rows} \times \text{columns} = 9 \times 36 = 324$$

- Space for four arrays representation

Array	Size
Next	71
Check	71
Default	9
Base	9
Total	160

- If a large graph seen as adjacency matrix is stored using four arrays, it would have the need of pointers and dynamic memory allocation

This would imply good cache behaviour

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



Further Compression Using Equivalence Classes

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

- The four arrays handle similarity in the rows of the 2-D table
- Several columns could have a lot of similarity too
- We can define equivalence classes of characters that have identical transitions
Identical columns are collapsed into a single column
- The equivalence classes are given contiguous codes thereby eliminating several “holes” in the Next and Check arrays



Further Compression Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

```
int nextstate(s,c)
{ if (Check[Base[s]+c] == s)
  return Next[Base[s]+c];
  else
    return nextstate(Default[s],c);
}
```

Now c represents the class of a character instead of the character



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0		
1		
2		
3		
4		
5		
6		
7		
8		

	N	C
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0		
1		
2		
3		
4		
5		
6		
7		
8		

	N	C
0	8	8
1	8	8
2	8	8
3	8	8
4	8	8
5	8	8
6	8	8
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0	8	•
1		
2		
3		
4		
5		
6		
7		
8		•

	N	C
0	8	8
1	8	8
2	8	8
3	8	8
4	8	8
5	8	8
6	8	8
7	1	0
8		
9		
10		
11		
12		
13	7	0
14	6	0
15		
16		



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0	8	●
1	8	●
2		
3		
4		
5		
6		
7		
8		●

N	C
0	8
1	8
2	8
3	8
4	8
5	8
6	8
7	1
8	2
9	
10	
11	
12	
13	7
14	6
15	
16	



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0	8	●
1	8	●
2	8	●
3		
4		
5		
6		
7		
8		●

N	C
0	8
1	8
2	8
3	8
4	8
5	8
6	8
7	1
8	2
9	3
10	
11	
12	
13	7
14	6
15	
16	



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check

S	D	B
0	8	●
1	8	●
2	8	●
3	8	●
4		
5		
6		
7		
8		●

N	C
0	8
1	8
2	8
3	8
4	8
5	8
6	8
7	1
8	2
9	3
10	4
11	
12	
13	7
14	6
15	
16	



Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

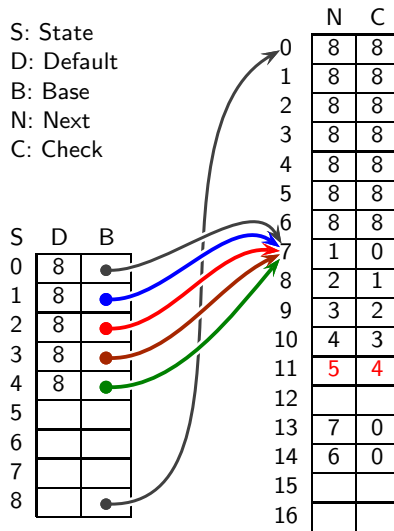
Tokenizing the Input

Representing DFAs

Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check





Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

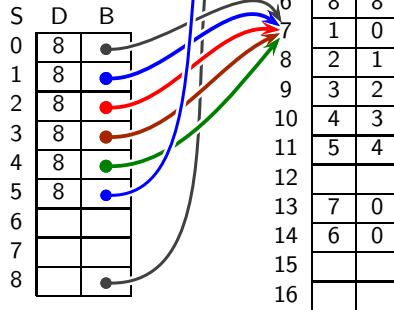
Tokenizing the Input

Representing DFAs

Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check





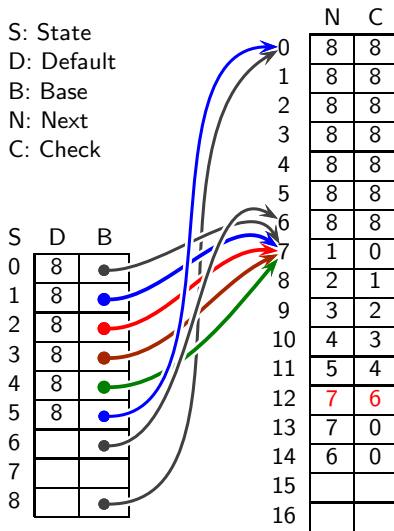
Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check





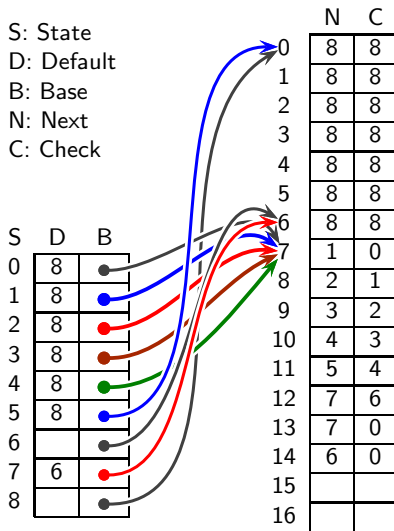
Four Arrays Representation Using Equivalence Classes for Example 2

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:
Scanning
Section:
Introduction
Specifying Scanners
Constructing DFAs
Tokenizing the Input
Representing DFAs
Minimizing DFAs

EC →	0	1	2	3	4	5	6	7
	b	e	g	i	n	L	D	-
0	1	8	8	8	8	8	7	6
1	8	2	8	8	8	8	8	
2	8	8	3	8	8	8	8	
3	8	8	8	4	8	8	8	
4	8	8	8	8	5	8	8	
5	8	8	8	8	8	8	8	
6							7	
7							7	
8	8	8	8	8	8	8	8	

S: State
D: Default
B: Base
N: Next
C: Check





IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

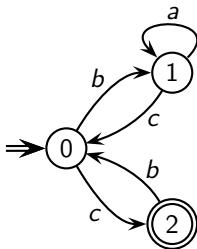
Tokenizing the Input

Representing DFAs

Minimizing DFAs

Tutorial Problem

Represent the following DFA using 4-arrays notation as compactly as possible



Character	Code
<i>a</i>	0
<i>b</i>	1
<i>c</i>	2



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

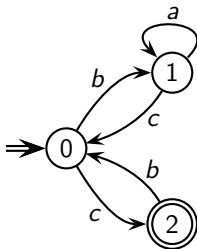
Tokenizing the Input

Representing DFAs

Minimizing DFAs

Tutorial Problem

Represent the following DFA using 4-arrays notation as compactly as possible



Character	Code
<i>a</i>	0
<i>b</i>	1
<i>c</i>	2

State	Base	Default
0	2	
1	0	
2	0	

	Next	Check
0	1	1
1	0	2
2	0	1
3	1	0
4	2	0
5		



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs



IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Minimizing DFAs



Minimizing DFAs

IIT Bombay
cs302: Implementation
of Programming
Languages

Topic:

Scanning

Section:

Introduction

Specifying Scanners

Constructing DFAs

Tokenizing the Input

Representing DFAs

Minimizing DFAs

Prof. Sanyal's slides (scanning-slides-sanyal-part3.pdf)